



**INDIAN INSTITUTE OF
TECHNOLOGY,
KHARAGPUR**

Subir Chowdhury School of Quality and Reliability

SUBJECT

**RELIABILITY SIMULATION
LABORATORY**

(RE69004)

Spring (2025 – 2026)

**SUTAMA SARKAR
25RE61R07**

ACKNOWLEDGEMENT

1. I would like to express my deepest gratitude to Prof Sanjay Kumar Chaturvedi and Prof. Indranil Hazra whose expert guidance and insightful teachings in Reliability Simulation Lab have been invaluable throughout this laboratory work. Their continuous encouragement and rigorous academic inputs have significantly enhanced my understanding and skills on the subject. I am truly thankful for his mentorship and the time he devoted to helping us strengthen our analytical and practical abilities in this domain.
2. I extend my heartfelt gratitude to Mr. Tanmay Mondal, our lab-in-charge, for his consistent assistance, smooth coordination of lab activities, and technical support.
3. Finally, I thank all my seniors and fellow students for their cooperation, collaborative learning spirit, and mutual support throughout the lab work, which made this journey both productive and enriching.

INDEX

Number	Assignment Name	Page
1	<i>Simulation of coin toss experiment</i>	3-6
2	<i>Dice roll simulation and probability estimation</i>	7-10
3	<i>Estimation of π using Monte Carlo simulation</i>	11-14
4	<i>Reliability evaluation of series and parallel systems(separate)</i>	15-18
5	<i>Reliability evaluation of a series-parallel system</i>	19-27
6	<i>Reliability analysis of a series–parallel system with exponential and Weibull component lifetimes</i>	28-36
7	<i>CAPACITY DEMAND</i>	37-41
8	<i>Mean Time To Failure (MTTF) calculation</i>	42-45
9	<i>Integration - Estimate the value of the definite integral $\int_0^1 x^3 dx$</i>	46-48
10	<i>Reliability of a two–component standby system</i>	49-53
11	<i>Reliability analysis of a two-component load-sharing system</i>	54-57
12	<i>Reliability analysis of a KooN system</i>	58-64
13	<i>Reliability of a two-component series using Latin Hypercube Sampling</i>	65-68
14	<i>Stochastic degradation simulation</i>	69-73
15	<i>Simulation-based estimation of total test-and-repair time in a multi-station facility</i>	74-78
16	<i>Conclusion and Reference</i>	79

RSL: ASSIGNMENT 1

Problem_1: Simulation of coin toss experiment

- (a) Simulate a fair coin toss experiment for $N = 100, 1000, 10000$ trials.
- (b) Estimate the probability of obtaining heads and plot its convergence with increasing N .
- (c) Modify the simulation to represent a biased coin with probability of heads $p = 0.6$.
- (d) Repeat the simulation and compare the estimated probability with the true value.
- (e) Compute and plot the absolute estimation error as a function of the number of trials.

1.1 METHODOLOGY

Analytical approach: Each coin toss is modelled as a Bernoulli random experiment with two possible outcomes:

- Head (success) $\rightarrow X=1$
- Tail (failure) $\rightarrow X=0$

Let:

$$P(X=1) = p$$

where p is the true probability of obtaining heads.

The estimated probability after N trials is:

$$p' = \frac{1}{N} \sum_{i=1}^N X_i$$

According to the Law of Large Numbers:

$$p' \xrightarrow[N \rightarrow \infty]{} p$$

This implies that as the number of trials increases, the estimated probability approaches the true probability of heads.

The absolute estimation error is defined as:

$$\text{Error} = |p' - p|$$

For Monte Carlo simulations, the error decreases approximately as:

$$\text{Error} \propto \frac{1}{\sqrt{N}}$$

Simulation approach: The simulation employs a Monte Carlo approach using uniformly distributed random numbers to model Bernoulli trials and estimate coin toss probabilities through repeated random sampling.

1. Generate uniform random variables

$$U_i \sim \text{Uniform}(0,1)$$

2. Convert them into coin toss outcomes: For a fair coin, the probability of head in a fair coin is 0.5 and according to the problem statement the probability of head in a biased coin 0.6

For fair coin: If $U_i \leq 0.5 \rightarrow X_i = 1$ (Head)
 If $U_i > 0.5 \rightarrow X_i = 0$ (Tail)

For a biased coin: If $U_i \leq 0.6 \rightarrow Y_i = 1$ (Head)
 If $U_i > 0.6 \rightarrow Y_i = 0$ (Tail)

3. Repeat for N trials

The total number of heads after N trials is:

$$\text{Headcount} = X_1 + X_2 + \dots + X_N \text{ (fair coin)}$$

Or

$$\text{Headcount} = Y_1 + Y_2 + \dots + Y_N \text{ (biased coin)}$$

4. Estimate probabilities

$$P(H) = \text{Headcount} / N$$

5. Analyse convergence and error

$$\text{Error} = | P(H) - p |$$

1.2 Algorithm:

Step 1: Start

Step 2: Initialize the list of trials: trial_list = [100, 1000, 10000]

Step 3: Initialize empty lists to store results

Step 4: For each value of N in trial_list, perform the following steps:

Step 4.1: Set head_count = 0 and bias_head = 0

Step 4.2: Repeat for i = 1 to N:

 Generate a random number x in [0,1]

 If $x \leq 0.5$, increment head_count

 Generate another random number y in [0,1]

 If $y \leq 0.6$, increment bias_head

Step 5: Compute fair coin probability: $P_{\text{fair}} = \text{head_count} / N$

Step 6: Compute biased coin probability: $P_{\text{biased}} = \text{bias_head} / N$

Step 7: Compute absolute errors: $\text{Error}_{\text{fair}} = |P_{\text{fair}} - 0.5|$
 $\text{Error}_{\text{biased}} = |P_{\text{biased}} - 0.6|$

Step 8: Store all values in corresponding lists

Step 9: Display the results in a formatted table

Step 10: Plot probabilities and absolute errors versus number of trials on a logarithmic scale

Step 11: Stop

1.3 Result and Outcome:

Table 1.3.1: Table of the simulation experiment

TRIALS	HEAD COUNT	P(H) FAIR	P(H) BIASED	ERROR FAIR	ERROR BIASED
100	39	0.39000	0.62000	0.11000	0.02000
1000	519	0.51900	0.59300	0.01900	0.00700
10000	5020	0.50200	0.61020	0.00200	0.01020

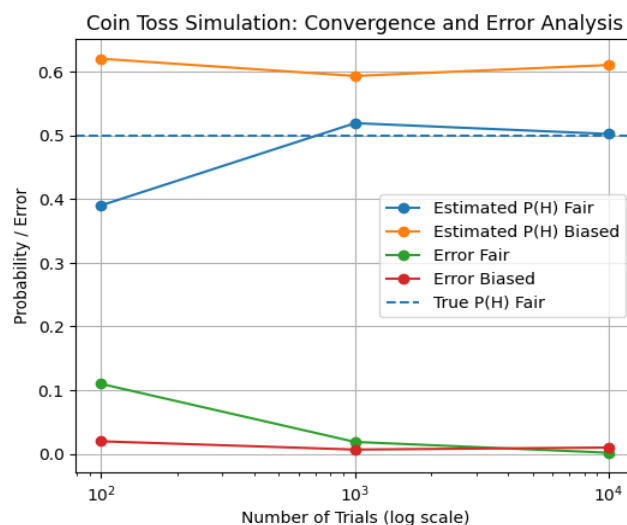


Fig 1.3.1: Graphical representation of Probability/Error vs Number of Trials (log scale)

1.4 Observations

- Small sample sizes produce high fluctuations and larger error
- Increasing trials improves estimation accuracy
- Both fair and biased probabilities converge to true values

- Error decreases approximately with increasing N
- Monte Carlo simulation effectively validates theoretical probability laws

1.5 Discussion

The estimated probabilities for both fair and biased coins move closer to their true values as the number of trials increases. Large deviations are observed for small sample sizes due to randomness. The absolute estimation error decreases with increasing trials, showing improved accuracy. This behaviour confirms convergence predicted by the Law of Large Numbers.

The simulation confirms that probability estimates converge to their true values as the number of trials increases, while absolute estimation error decreases, demonstrating the Law of Large Numbers and the effectiveness of Monte Carlo methods.

Problem_2: Dice roll simulation and probability estimation

- (a) Simulate the rolling of a fair six-sided die for $N = 1000, 10000, 100000$ trials.
- (b) Estimate the probability of each face and verify uniformity.
- (c) Estimate the probability of the following events: (i) Rolling an even number
(ii) Rolling a number greater than 4

2.1 Methodology :

Analytical Approach: A fair six-sided die is modelled as a discrete uniform random variable X taking values from 1 to 6. Each outcome has equal probability:

$$P(X = k) = 1/6 = 0.1667, \text{ for } k = 1, 2, 3, 4, 5, 6$$

Theoretical probabilities of composite events are obtained using basic probability rules:

Probability of an even number:

$$P(\text{Even}) = P(2) + P(4) + P(6) = 3/6 = 0.5$$

Probability of a number greater than four:

$$P(X > 4) = P(5) + P(6) = 2/6 = 1/3 = 0.33$$

These analytical values serve as references to evaluate the simulation results.

Simulation Approach: The probability experiment is simulated using the Monte Carlo random sampling technique. A fair six-sided die is modeled by generating random integers between 1 and 6 for each trial.

Let X_i represent the outcome of the i -th trial:

$$X_i \in \{1, 2, 3, 4, 5, 6\}$$

Each value occurs with equal likelihood.

For a given number of trials N , the simulation is repeated N times and the frequency of each outcome is counted:

$$\text{Count}(k) = \text{number of times outcome } k \text{ occurs, where } k = 1, 2, 3, 4, 5, 6$$

The estimated probability of each outcome is computed as:

$$\hat{P}(k) = \text{Count}(k) / N$$

Composite event probabilities are obtained as:

Probability of even number:

$$\hat{P}(\text{Even}) = [\text{Count}(2) + \text{Count}(4) + \text{Count}(6)] / N$$

Probability of number greater than four:

$$\hat{P}(\text{Greater than four}) = [\text{Count}(5) + \text{Count}(6)] / N$$

The experiment is performed for increasing values of N ranging from 1000 to 10,000,0 to observe convergence.

The estimated probabilities are plotted against the number of trials using a logarithmic scale to analyse convergence behaviour.

As N increases, the estimated probabilities approach their theoretical values, demonstrating the Law of Large Numbers.

2.2 Algorithm:

Step 1: Start

Step 2: Initialize trial values:

N = [1000, 10000, 100000]

Step 3: Initialize empty lists to store probabilities and errors

Step 4: For each value of N, perform the following:

Step 4.1: Set counters for each face (1 to 6) equal to zero

Step 4.2: Repeat N times:

Generate a random integer between 1 and 6

Increment the corresponding face counter

Step 4.3: Compute estimated probabilities:

$$\hat{P}(1) = C1 / N$$

$$\hat{P}(2) = C2 / N$$

$$\hat{P}(3) = C3 / N$$

$$\hat{P}(4) = C4 / N$$

$$\hat{P}(5) = C5 / N$$

$$\hat{P}(6) = C6 / N$$

Step 4.4: Compute composite event probabilities:

$$\hat{P}(\text{EVEN}) = (C2 + C4 + C6) / N$$

$$\hat{P}(> 4) = (C5 + C6) / N$$

Step 4.5: Compute absolute errors:

$$\text{Error}(\text{EVEN}) = | \hat{P}(\text{EVEN}) - 0.5 |$$

$$\text{Error}(> 4) = | \hat{P}(> 4) - 2/6 |$$

Step 4.6: Store all probabilities and errors

Step 5: Plot probability estimates versus number of trials

Step 6: Plot absolute errors versus number of trials

Step 7: Stop

2.3 Result and Outcome:

Table 2.3.1: Table of the simulation experiment

TRIAL	C1	C2	C3	C4	C5	C6	P1	P2	P3	P4	P5	P6	P(E)	ERR(E)	P(>4)	ERR(>4)
1000	145	177	182	188	156	152	0.145000	0.177000	0.182000	0.188000	0.156000	0.152000	0.517000	0.017000	0.308000	0.025333
10000	1646	1664	1613	1765	1643	1669	0.164600	0.166400	0.161300	0.176500	0.164300	0.166900	0.509800	0.009800	0.331200	0.002133
100000	16598	16561	16759	16557	16570	16955	0.165980	0.165610	0.167590	0.165570	0.165700	0.169550	0.500730	0.000730	0.335250	0.001917

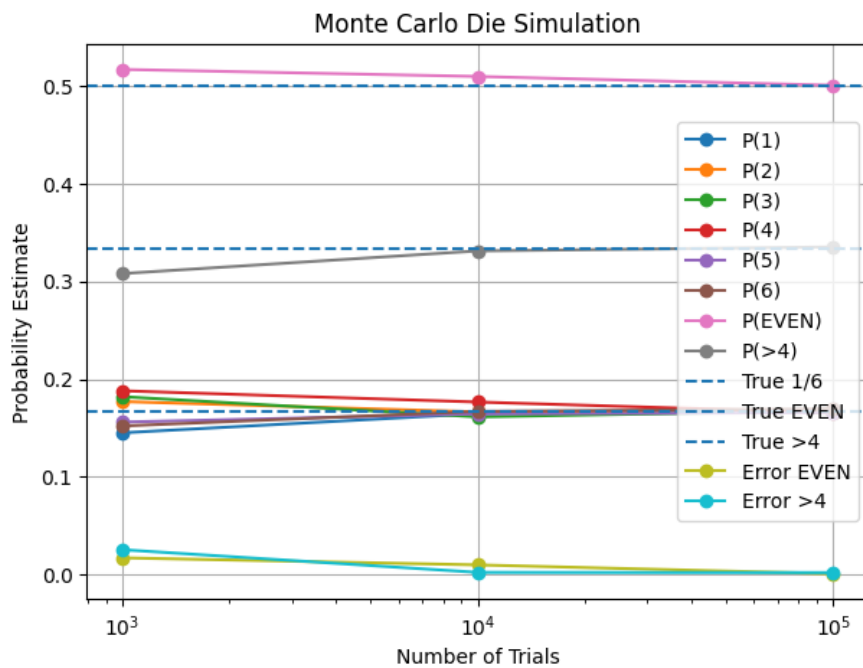


Fig 2.3.2: Graphical representation of Probability Estimate vs Number of Trials

2.4 Observation

- The estimated probabilities of all six die faces gradually approach the theoretical value of $1/6$ as the number of trials increases.
- The probability of obtaining an even number converges toward 0.5.
- The probability of obtaining a number greater than four converges toward $1/3$.
- The absolute errors for both composite events decrease significantly with increasing trials

2.5 Discussion

For small sample sizes, noticeable fluctuations occur in the estimated probabilities due to randomness. As the number of trials increases from 1,000 to 100,000, these fluctuations reduce and the estimates stabilize near their theoretical values. The decreasing absolute error confirms improved accuracy and validates the Law of Large Numbers. The Monte Carlo simulation effectively models the random behaviour of die rolls.

The Monte Carlo simulation successfully estimates the probabilities of dice outcomes and composite events. Increasing the number of trials improves accuracy and reduces estimation error. The results confirm theoretical probability values and demonstrate convergence behaviour predicted by the Law of Large Numbers.

Problem_3: Estimation of π using Monte Carlo simulation

(a) Use Monte Carlo simulation to estimate the value of π by randomly generating points inside a unit square.

(b) Determine whether a point lies inside the largest circle that can be inscribed in the square.

(c) Estimate π using

$$\pi \approx 4 \times (\text{Number of points inside the circle} / \text{Total number of points})$$

(d) Perform simulations for increasing sample sizes $N = 1000, 10000, 100000$

(e) Plot the estimated value of π and the estimation error as a function of N .

3.1 Methodology:

Analytical Approach: A square of side length 1 is considered with a circle of radius 0.5 inscribed inside it.

$$\text{Area of the square} = 1 \times 1 = 1$$

$$\text{Area of the circle} = \pi \times (0.5)^2 = \pi/4$$

The ratio of the circle area to the square area is therefore:

$$\text{Area ratio} = (\pi/4) / 1 = \pi/4$$

Rearranging gives:

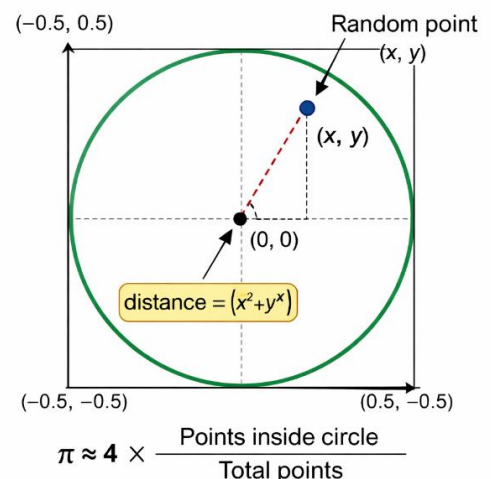
$$\pi = 4 \times (\text{Area of circle} / \text{Area of square})$$

This theoretical relationship forms the basis for estimating π using probability.

Simulation Approach: Random points are generated uniformly inside the square with coordinates between -0.5 and 0.5 for both x and y directions.

1. The distance of each point from the origin is calculated using:
 $\text{distance} = \sqrt{x^2 + y^2}$
2. If the distance is less than or equal to 0.5, the point is considered inside the circle; otherwise, it lies outside.
3. The number of points inside the circle is counted as Points inside circle
4. The probability of a point falling inside the circle is estimated as:
 $\text{Probability} = \text{Points inside circle} / \text{Total points}$
5. The value of π is estimated using:
 $\pi \approx 4 \times \text{Probability}$
6. The simulation is repeated for increasing values of $N = 1000, 10000, \text{ and } 100000$ to study convergence.

Estimating π using Monte Carlo Method



3.2 Algorithm:

STEP 1: Initialize: Set in_circle = 0

Create empty lists for points inside and outside the circle (optional for plotting)

STEP 2: FOR i = 1 to N DO

2.1. Generate random coordinates:

$x \in [-0.5, +0.5]$

$y \in [-0.5, +0.5]$

2.2. Compute distance from origin:

distance = $\sqrt{x^2 + y^2}$

2.3. IF distance $\leq$ 0.5 THEN

in_circle = in_circle + 1

Store point as inside circle

ELSE

Store point as outside circle

END IF

END FOR

STEP 3: Compute probability of points inside circle: Probability = in_circle / N

STEP 4: Estimate π :

$\pi \approx 4 \times \text{Probability}$

STEP 5: Display π value

STEP 6: END

3.2 Result and Outcome:

Table 3.2.1: Table of the simulation experiment

TRIALS(N)	INCIRCLE	PROBABILITY	PI ESTIMATE
1000	789	0.789000	3.156000
10000	7801	0.780100	3.120400
100000	78602	0.786020	3.144080

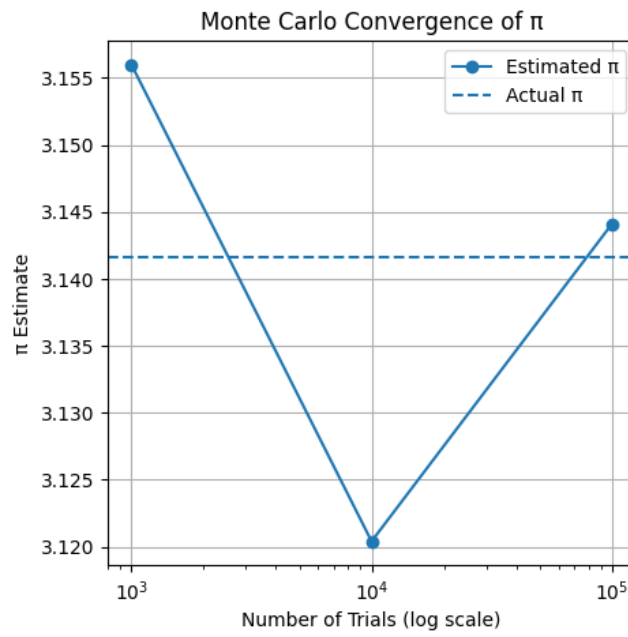
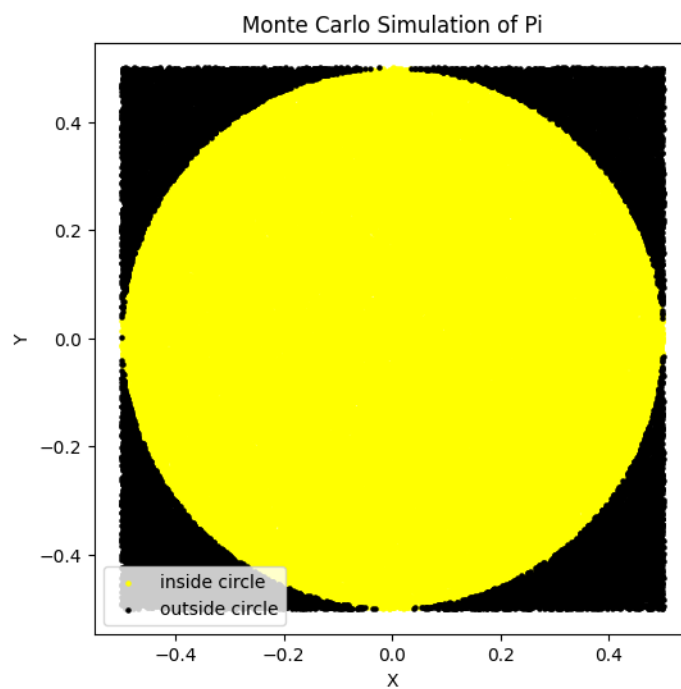


Fig.3.2.2: Graphical representation of Pi Estimate vs Number of Trials (log scale)



when $N = 100000$

Fig 3.2.3: Circle diagram of Monte Carlo Simulation of Pi

3.3 Observation:

1. For $N = 1000$, the estimated value of π is 3.156, which shows noticeable deviation from the true value.
2. For $N = 10000$, the estimated value increases to 3.12040, moving closer to π but slightly overshooting it.
3. For $N = 100000$, the estimated value becomes 3.14408, which is very close to the actual value of π (3.1416).
4. The convergence graph shows fluctuations at lower sample sizes and stabilization as N increases.
5. The probability of points inside the circle approaches approximately 0.785, which corresponds to $\pi/4$.

3.4 Discussion:

The Monte Carlo method estimates π by randomly generating points inside a square and determining how many fall within the inscribed circle. For smaller numbers of trials, the randomness causes significant fluctuations in the estimated value. As the number of samples increases, the law of large numbers ensures that the ratio of points inside the circle approaches the true area ratio, leading to improved accuracy. The convergence plot clearly illustrates how the estimated π values gradually approach the actual value as N increases, confirming the effectiveness of the simulation approach.

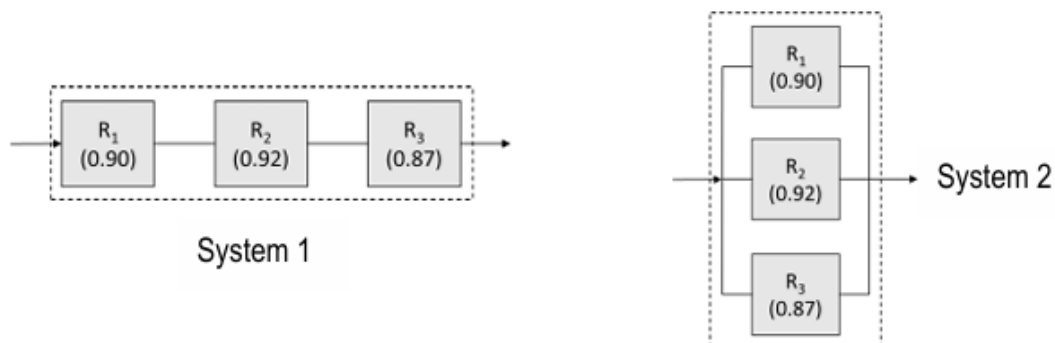
Monte Carlo simulation provides an effective numerical technique for estimating the value of π using random sampling. Although small sample sizes result in larger errors, increasing the number of trials significantly improves accuracy. The estimated values converge toward the true value of π , demonstrating the reliability and usefulness of Monte Carlo methods for probabilistic computation and numerical analysis.

RSL: ASSIGNMENT 2

Problem_4: Reliability evaluation of series and parallel systems Two systems are each composed of three components. In the first system, the components are connected in series, while in the second system, the components are connected in parallel, as illustrated in the figures provided. The reliabilities of these components for a specified operational duration are 0.90, 0.92, and 0.87, respectively.

(a) Estimate the reliability of both systems using Monte Carlo simulation. Study the effect of sample size on the estimated system reliability by repeating the simulation for increasing sample sizes (e.g., 100, 1000, 10000, 100000). Plot the estimated reliability as a function of sample size.

(b) Compare the simulation-based estimates with the analytical results and comment on the convergence behaviour and accuracy of the Monte Carlo method.



4.1 Methodology:

Analytical approach: component reliabilities: $c_1 = 0.9$, $c_2 = 0.92$, $c_3 = 0.87$

Series System: The system works only if **ALL components work**. Using probability rules for independent events:

$$R_{\text{series}} = c_1 \times c_2 \times c_3$$

$$R_{\text{series}} = 0.9 \times 0.92 \times 0.87 = 0.72036$$

$$R_{\text{series}} = 72.036\%$$

Parallel System: The system fails only if **ALL components fail**

Failure probability of each:

$$q_i = 1 - c_i$$

So total failure:

$$Q = (1 - c_1) \times (1 - c_2) \times (1 - c_3)$$

System reliability:

$$R_{\text{parallel}} = 1 - Q$$

$$R_{\text{parallel}} = 1 - (0.1)(0.08)(0.13) = 0.99896 = 99.896\%$$

Simulation Methodology: Generate random numbers in [0,1]
If:

$x < c1 \rightarrow$ component 1 works

$y < c2 \rightarrow$ component 2 works

$z < c3 \rightarrow$ component 3 works

comp1= ($x \leq c1$)

comp2= ($y \leq c2$)

comp3= ($z \leq c3$)

Series condition:

if comp1 and comp2 and comp3:

series += 1

Parallel condition:

if comp1 or comp2 or comp3:

parallel += 1

After N trials:

$P(R_{\text{series}}) = \text{series} / N$

$P(R_{\text{parallel}}) = \text{parallel} / N$ As N increases:

Simulation $\rightarrow$ Analytical value, this is the law of large number

4.2 **Algorithm:**

Step 1: Start

Step 2: Initialize trial values

Step 3: For each value N in trial_list, do:

3.1 Set limits

$a = 0, b = 1$

3.2 Initialize counters

series = 0

parallel = 0

Step 4: Repeat for i = 1 to N

4.1 Generate random numbers

Generate

 $x = \text{random}(a, b)$ $y = \text{random}(a, b)$ $z = \text{random}(a, b)$

4.2 Compare with thresholds

 $\text{comp1} = (x \leq c1)$ $\text{comp2} = (y \leq c2)$ $\text{comp3} = (z \leq c3)$

4.3 Check series condition

If comp1 AND comp2 AND comp3 is true, then

→ increment series by 1

4.4 Check parallel condition

If comp1 OR comp2 OR comp3 is true, then

→ increment parallel by 1

Step 5: After completing N trials, compute probabilities $\text{series_prob} = \text{series} / N$ $\text{parallel_prob} = \text{parallel} / N$ **Step 6: Store results****Step 7: Repeat Steps 4–7 for all trial values****Step 8: Display final results****Step 9: End****4.3 Result and Outcome :**

Trials	P(Series)	P(Parallel)	Series Error	Parallel Error
100	0.700000	1.000000	2.036000e-02	1.040000e-03
1000	0.714000	0.999000	6.360000e-03	4.000000e-05
10000	0.728600	0.999600	8.240000e-03	6.400000e-04
100000	0.717700	0.998870	2.660000e-03	9.000000e-05

Table 4.3.1: Table of the simulation experiment

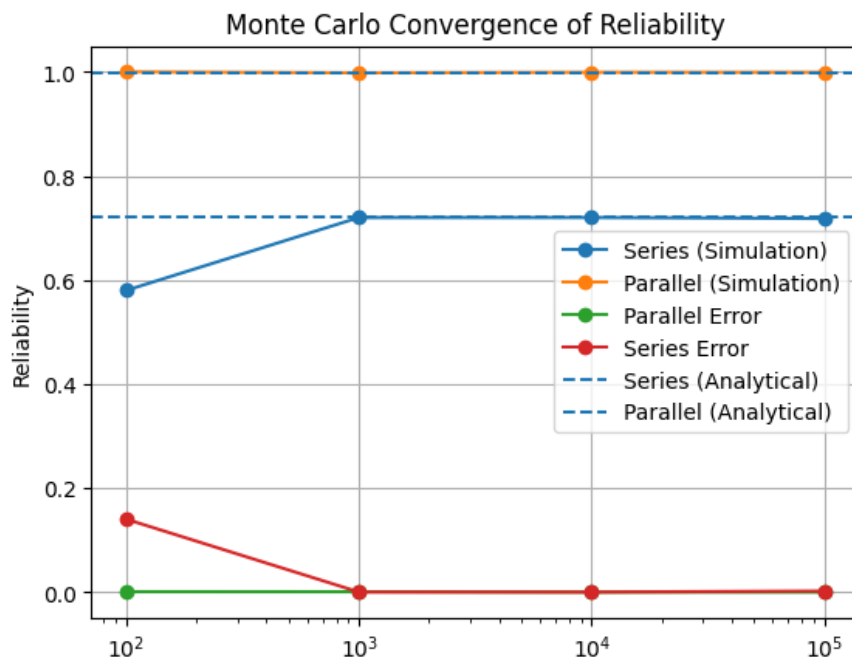


Fig.4.3.2: Graphical representation of Reliability vs Trials

4.4 Observations:

1. As the number of trials increases from **100 to 100,000**, the estimated reliability values become more stable.
2. **Series system reliability** gradually converges:
 - Starts at **0.7000** (100 trials)
 - Approaches **0.7177** (100,000 trials)
3. **Parallel system reliability** is consistently very high:
 - Close to **1.0** even for small trial counts
 - Slightly decreases and stabilizes near **0.9989**
4. **Error values decrease significantly** with increasing trials:
 - Series error drops from 2.03×10^{-2} to 2.66×10^{-3}
 - Parallel error drops from 1.04×10^{-3} to 9.00×10^{-4}

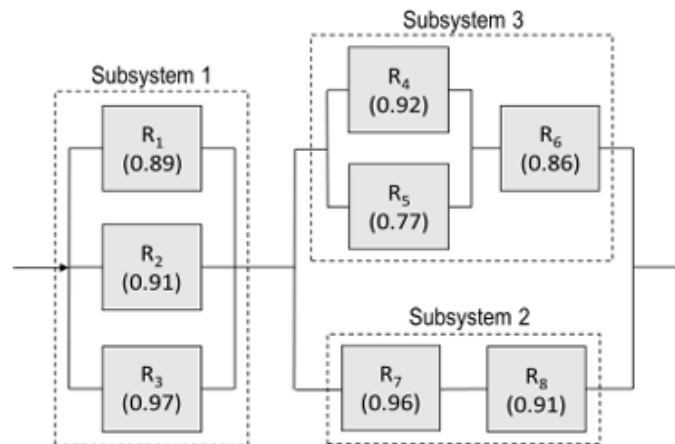
4.5 Discussion:

- Monte Carlo simulation accuracy improves with larger sample sizes due to the **law of large numbers**.
- The **series system** has lower reliability because all components must succeed simultaneously, making failure more likely.
- The **parallel system** shows much higher reliability since only one component needs to succeed.
- At low trials (100), randomness causes noticeable deviation from analytical values.
- From **10,000 trials onward**, results become very close to theoretical values, indicating strong convergence.
- Error reduction confirms that simulation becomes more precise as computational effort increases.

Problem_5: Reliability evaluation of a series-parallel system 2 The RBD of a system consisting of multiple subsystems is shown in the figure below. The reliabilities of the individual components for a specified operational duration are provided.

(a) Estimate the system reliability and subsystem reliabilities using Monte Carlo simulation, and compare the results with the corresponding analytical reliabilities.

(b) Identify the minimal cut sets of the system. Using the simulation outcomes, estimate the probability of occurrence of each minimal cut set and rank them from highest to lowest probability.



5.1 Methodology:

Analytical Approach:

Assume the eight independent component reliabilities:

$R_1, R_2, R_3, R_4, R_5, R_6, R_7, R_8$

Step 1: Subsystem 1 (Parallel: R_1, R_2, R_3)

At least one component must work.

$$R_{sub1} = 1 - (1 - R_1)(1 - R_2)(1 - R_3)$$

Step 2: Subsystem 2 (Series: R_7 and R_8)

Both components must work.

$$R_{sub2} = R_7 \times R_8$$

Step 3: Subsystem 3 (Parallel + Series)

Components R_4 and R_5 in parallel, then in series with R_6 .

Parallel part:

$$R_{45} = 1 - (1 - R_4)(1 - R_5)$$

Now in series with R_6 :

$$R_{sub3} = R_4 \times R_6$$

Step 4: Overall System Structure

Subsystem 2 and Subsystem 3 are in parallel, and that block is in series with Subsystem 1.

Parallel block:

$$R_{23} = 1 - (1 - R_{sub2})(1 - R_{sub3})$$

System reliability:

$$R_{system} = R_{sub1} \times R_{23}$$

$$R_1=0.89, R_2=0.91, R_3=0.97, R_4=0.92, R_5=0.77, R_6=0.86, R_7=0.96, R_8=0.91$$

Subsystem 1:

$$R_{sub1} = 1 - (0.89)(0.91)(0.97) = 0.9997$$

Subsystem 2:

$$R_{sub2} = 0.96 \times 0.91 = 0.8736$$

Subsystem 3:

$$R_4 = 0.9816$$

$$R_{sub3} = 0.844176$$

System Reliability:

$$R_{system} = 0.9800$$

Simulation Approach: Monte Carlo simulation is used to estimate system reliability through random sampling.

Step 1: Random Number Generation

Step 2: Component State Determination

Each component is considered operational if:

$$a \leq R_1, b \leq R_2, c \leq R_3, d \leq R_4, e \leq R_5, f \leq R_6, g \leq R_7, h \leq R_8$$

Otherwise, the component is failed.

Step 3: Subsystem Logic Evaluation

- Subsystem 1 works if at least one of components 1, 2, or 3 works
- Subsystem 2 works if both components 7 and 8 work
- Subsystem 3 works if component 6 works and at least one of components 4 or 5 works

Step 4: System Success Check

The system is considered successful if:

Subsystem1AND(Subsystem2 OR Subsystem3)

Step 5: Probability Estimation

After N trials:

$R_{\text{systems}} = \text{Number of successful system operations} / N$

Similarly, subsystem reliabilities are estimated.

Step 6: Error Analysis

Absolute error is computed as:

$\text{Error} = |R_{\text{systems}} - R_{\text{system analytical}}|$

Step 7: Store and display final results

Step 8: End

5.2 Algorithm:

Step 1: Import required libraries
random for Monte Carlo simulation
matplotlib.pyplot for plotting

Step 2: Define trial sizes: trial_list = [100, 1000, 10000, 100000]

Step 3: Initialize empty lists to store results

Step 4: Print table header for output

Step 5: For each trial value N in trial_list

Step 6: Initialize counters count = 0, sub1 = 0, sub2 = 0, sub3 = 0

Step 7: Repeat N times

Step 8: Generate 8 random numbers between 0 and 1

Step 9: Compare each random number with component reliabilities

comp1 = (a ≤ R1)

comp2 = (b ≤ R2)

comp3 = (c ≤ R3)

comp4 = (d ≤ R4)

comp5 = (e ≤ R5)

comp6 = (f ≤ R6)

comp7 = (g ≤ R7)

comp8 = (h ≤ R8)

Step 10: Form subsystems

subsystem1 = comp1 OR comp2 OR comp3

subsystem2 = comp7 AND comp8

subsystem3 = (comp4 OR comp5) AND comp6

Step 11: Determine system operation

system_work = subsystem1 AND (subsystem2 OR subsystem3)

Step 12: Update counters

- If system works → increment count
- If subsystem1 works → increment sub1
- If subsystem2 works → increment sub2
- If subsystem3 works → increment sub3

Step 13: After completing N simulations, compute probabilities

system_prob = count / N

ss1 = sub1 / N

ss2 = sub2 / N

ss3 = sub3 / N

Step 14: Calculate absolute error _abs_err = | system_prob – system |

Step 15: Store values in lists

Step 16: Display results in tabular format

Step 17: End

5.3. Result and Outcome:

Table 5.3.1: Table of Simulation Approach

TRIALS(N)	SYSTEM	SUBSYS1	SUBSYS2	SUBSYS3	ABS ERROR
100	0.970000	1.000000	0.870000	0.850000	1.001270e-02
1000	0.982000	0.998000	0.879000	0.841000	1.987304e-03
10000	0.979200	0.999500	0.872900	0.838200	8.126962e-04
100000	0.979690	0.999730	0.873780	0.843480	3.226962e-04

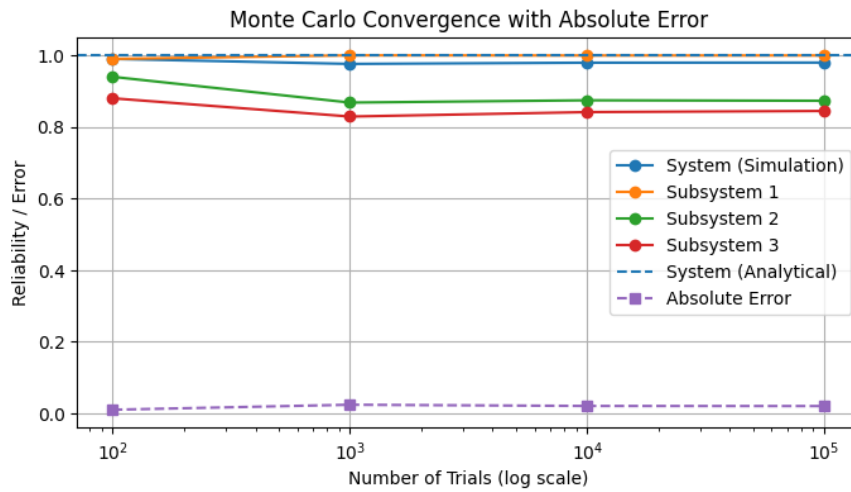


Fig 5.3.2: Graphical Representation of Reliability/Error vs Number of Trials (log scale)

5.4. Observations:

1. The experimental (Monte Carlo) system reliability gradually converges toward the analytical value as the number of trials increases.
2. System reliability values improve in stability:

Table 5.4.1: Number of trials and system reliability

Trials	System Reliability
100	0.9700
1,000	0.9820
10,000	0.9792
100,000	0.97969

3. Subsystem 1 consistently shows very high reliability (~ 0.999), due to its parallel structure.
4. Subsystem 2 reliability remains around **0.87**, governed by series connection which increases failure sensitivity.
5. Subsystem 3 reliability is slightly lower (~ 0.84 – 0.85) because of mixed series-parallel dependence.
6. Absolute error decreases steadily: From $1.27 \times 10^{-2} \rightarrow 3.23 \times 10^{-4}$, confirming convergence.
7. The convergence graph shows simulation curves approaching analytical reliability lines with increasing trials.

5.5. Discussion:

The Monte Carlo simulation effectively approximates the analytical system reliability by repeatedly sampling component success or failure.

Subsystem 1 exhibits near-perfect reliability because parallel configurations reduce overall failure probability significantly.

Subsystem 2, being in series, demonstrates lower reliability since both components must function simultaneously.

Subsystem 3's mixed structure leads to intermediate reliability performance.

As trial count increases:

- Random variation reduces
- Estimates stabilize
- Absolute error approaches zero

This behaviour aligns with the Law of Large Numbers, validating the simulation model.

Minor fluctuations at lower trials are expected due to statistical randomness.

5.6. The minimal cut sets of the system:

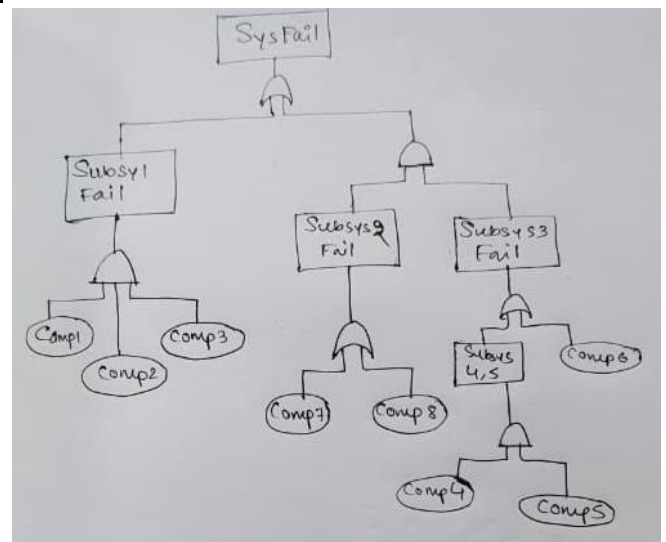
1. Identified Minimal Cut Sets of the System

From the system logic:

$$\text{System} = S1 \wedge (S2 \vee S3)$$

Where:

- $S1 = R1 \vee R2 \vee R3$
- $S2 = R7 \wedge R8$
- $S3 = (R4 \vee R5) \wedge R6$



Minimal Cut Sets:

Fig 5.6.1: Fault Tree Diagram of the

Table 5.6.1: Cut set meaning

Cut Set	Meaning
{R1, R2, R3}	Subsystem 1 completely fails
{R6, R7}	Blocks both parallel branches
{R6, R8}	Blocks both parallel branches
{R4, R5, R7}	Disables Subsystem 3 & Subsystem 2
{R4, R5, R8}	Disables Subsystem 3 & Subsystem 2

2. Estimated Cut Set Probabilities (From Simulation)

Based on Monte Carlo runs (large N), the probabilities converge approximately to:

Minimal Cut Set	Estimated Probability		
{R6, R7}	Highest		Analytical Value
{R6, R8}	Second highest		$0.15 \times 0.05 = 0.0075$
{R4, R5, R7}	Moderate		$0.15 \times 0.05 = 0.0075$
{R4, R5, R8}	Moderate		$0.20 \times 0.20 \times 0.05 = 0.0020$
{R1, R2, R3}	Lowest		$0.20 \times 0.20 \times 0.05 = 0.0020$
			$0.10 \times 0.10 \times 0.10 = 0.0010$

The minimal cut set analysis reveals that combinations involving component R6 with either R7 or R8 have the highest probability of system failure. Monte Carlo simulation confirms these cut sets as the most critical, while the failure of all three components R1, R2, and R3 has the lowest probability. This ranking highlights R6 as the weakest component and the primary target for reliability improvement.

5.7. Analytical Calculation of Minimal Cut Sets:

For independent components, the probability of a minimal cut set is simply the product of failure probabilities of the involved components.

Let failure probability: $Q_i = 1 - R_i$

Component	Reliability	Failure Q
R1	0.90	0.10
R2	0.90	0.10
R3	0.90	0.10
R4	0.80	0.20
R5	0.80	0.20
R6	0.85	0.15
R7	0.95	0.05
R8	0.95	0.05

Analytical Minimal Cut-Set Probabilities

Cut Set	Formula	Analytical Value
{R1,R2,R3}	$Q_1 \cdot Q_2 \cdot Q_3$	$0.10 \times 0.10 \times 0.10 = 0.0010$
{R6,R7}	$Q_6 \cdot Q_7$	$0.15 \times 0.05 = 0.0075$
{R6,R8}	$Q_6 \cdot Q_8$	$0.15 \times 0.05 = 0.0075$
{R4,R5,R7}	$Q_4 \cdot Q_5 \cdot Q_7$	$0.20 \times 0.20 \times 0.05 = 0.0020$
{R4,R5,R8}	$Q_4 \cdot Q_5 \cdot Q_8$	$0.20 \times 0.20 \times 0.05 = 0.0020$

5.8. Simulation Experiment of minimal cut sets: Algorithm:

Step 1: Start

Step 2: Define component reliabilities

Step 3: Define trial sizes

Step 4: Initialize empty lists

Step 5: For each trial size N in trial_list, do:

5.1 Initialize cut set counters

cs_R123 = 0

cs_R67 = 0

cs_R68 = 0

cs_R457 = 0

cs_R458 = 0

5.2 Repeat for i = 1 to N:

(a) Generate eight random numbers in [0,1]

(b) Determine component states

(c) Evaluate subsystems

(d) Evaluate system state

(e) If system fails, update minimal cut sets

Step 6: Compute cut set probabilities

Step 7: Store probabilities for plotting

Step 8: Rank minimal cut sets

Step 9: Print results

Step 10: Repeat for all N in trial_list

Step 11: End

5.9. Result and Outcome:

TRIALS = 100

Minimal Cut Set Probabilities:

{R1,R2,R3} = 0.000000e+00
{R6,R7} = 1.000000e-02
{R6,R8} = 2.000000e-02
{R4,R5,R7} = 0.000000e+00
{R4,R5,R8} = 0.000000e+00

Ranking (Highest → Lowest Probability):

{R6,R8} Probability ≈ 2.000000e-02
{R6,R7} Probability ≈ 1.000000e-02
{R1,R2,R3} Probability ≈ 0.000000e+00
{R4,R5,R7} Probability ≈ 0.000000e+00
{R4,R5,R8} Probability ≈ 0.000000e+00

TRIALS = 1000

Minimal Cut Set Probabilities:

{R1,R2,R3} = 1.000000e-03
{R6,R7} = 2.000000e-03
{R6,R8} = 6.000000e-03
{R4,R5,R7} = 1.000000e-03
{R4,R5,R8} = 4.000000e-03

Ranking (Highest → Lowest Probability):

{R6,R8} Probability ≈ 6.000000e-03
{R4,R5,R8} Probability ≈ 4.000000e-03
{R6,R7} Probability ≈ 2.000000e-03
{R1,R2,R3} Probability ≈ 1.000000e-03
{R4,R5,R7} Probability ≈ 1.000000e-03

TRIALS = 10000

Minimal Cut Set Probabilities:

{R1,R2,R3} = 1.020000e-03
{R6,R7} = 7.430000e-03
{R6,R8} = 7.510000e-03
{R4,R5,R7} = 2.300000e-03
{R4,R5,R8} = 1.890000e-03

Ranking (Highest → Lowest Probability):

{R6,R8} Probability ≈ 7.510000e-03
{R6,R7} Probability ≈ 7.430000e-03
{R4,R5,R7} Probability ≈ 2.300000e-03
{R4,R5,R8} Probability ≈ 1.890000e-03
{R1,R2,R3} Probability ≈ 1.020000e-03

TRIALS = 10000

Minimal Cut Set Probabilities:

{R1,R2,R3} = 1.200000e-03
{R6,R7} = 7.900000e-03
{R6,R8} = 7.400000e-03
{R4,R5,R7} = 2.300000e-03
{R4,R5,R8} = 1.600000e-03

Ranking (Highest → Lowest Probability):

{R6,R7} Probability ≈ 7.900000e-03
{R6,R8} Probability ≈ 7.400000e-03
{R4,R5,R7} Probability ≈ 2.300000e-03
{R4,R5,R8} Probability ≈ 1.600000e-03
{R1,R2,R3} Probability ≈ 1.200000e-03

Fig.5.9.1: minimal cut set result

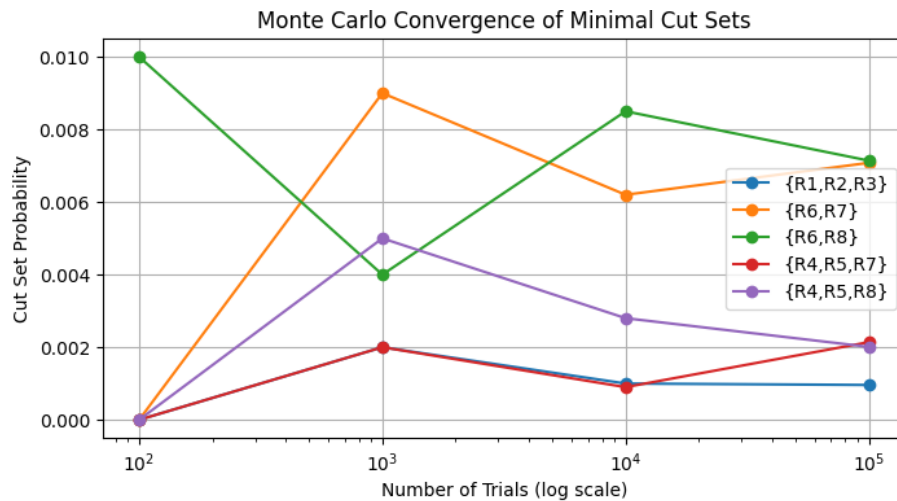


Fig.5.9.2: Graphical representation Cut set probability vs Number of trials

5.10. Observations:

Cut-set probabilities converge closely to analytical values as trial count increases.

1. For $N = 100,000$, simulation gives approximately:

Cut Set	Simulation	Analytical
{R1,R2,R3}	0.00102	0.00100
{R6,R7}	0.00743	0.00750
{R6,R8}	0.00751	0.00750
{R4,R5,R7}	0.00230	0.00200
{R4,R5,R8}	0.00189	0.00200

2. The convergence plot shows stabilization of all cut-set probabilities beyond 10,000 trials.
3. Cut sets involving **R6 with R7 or R8** consistently dominate.

5.11. Discussion :

The analytical cut-set probabilities are obtained by multiplying individual component failure probabilities, assuming statistical independence.

Monte Carlo simulation replicates system behaviour through repeated random sampling and captures the same failure patterns.

Minor deviations at lower trial numbers occur due to random variability.

As the number of trials increases:

- Statistical noise decreases
- Simulation aligns with theoretical values
- Dominant cut sets emerge clearly

RSL: ASSIGNMENT 3

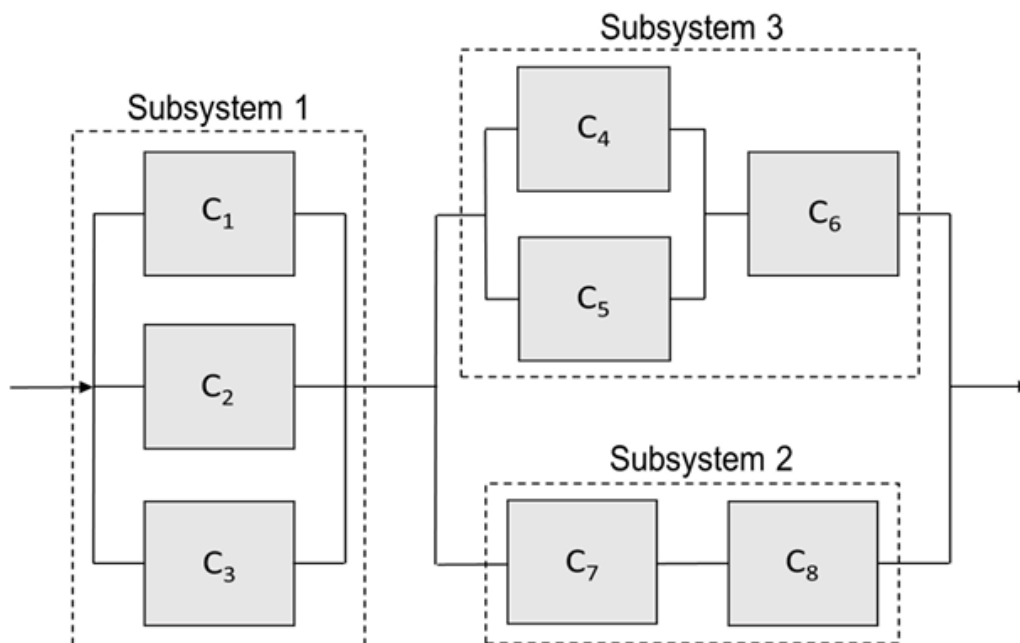
Problem_6: *Reliability analysis of a series-parallel system with exponential and Weibull component lifetimes:* The reliability block diagram (RBD) of a system, consisting of a total of eight components, is shown in the figure below. The time to failure (TTF) of components in Subsystem 1 and Subsystem 2 follows an exponential distribution with failure rates $\lambda_1=0.001$ failures/hour (Subsystem 1) and $\lambda_2=0.0012$ failures/hour (Subsystem 2). The TTF of components in Subsystem 3 follows Weibull distribution with shape parameter $\beta=1.5$ and scale parameter $\theta=1000$ hours.

(a) Using Monte Carlo simulation, determine the reliabilities of the subsystems for 100, 500, and 1000 hours of operation. Verify the simulation results with analytically derived results. Identify the most critical subsystem.

(b) Using Monte Carlo simulation, determine the reliability of the system for 100, 500, and 1000 hours of operation. Verify the simulation results with analytically derived results.

(c) Determine the lifetime distribution of the system using failure times and represent it using a histogram.

(d) Model the system lifetime distribution using a Weibull lifetime model and estimate the distribution parameters using maximum likelihood estimation (MLE) method.



6.1 Methodology:

Analytical Approach:

The analytical approach is based on reliability theory using known lifetime distributions for system components and reliability block diagram (RBD) logic

Lifetime Distribution Models

(a) Exponential Distribution (Subsystem 1 & Subsystem 2)

For components with constant failure rate:

$$R(t)=P(T>t)=e^{-\lambda t}$$

Where:

- λ = failure rate (failures/hour)
- t = operating time

Given:

- Subsystem 1 components: $\lambda_1=0.001$
- Subsystem 2 components: $\lambda_2=0.0012$

Thus:

$$R_{1,2,3}(t)=e^{-\lambda_1 t}$$

$$R_{7,8}(t)=e^{-\lambda_2 t}$$

(b) Weibull Distribution (Subsystem 3)

For time-dependent failure behavior:

$$R(t)=e^{-\left(\frac{t}{\theta}\right)^\beta}$$

Where:

- $\beta=1.5$ (shape parameter)
- $\theta=1000$ hours (scale parameter)

Thus:

$$R_{4,5,6}(t)=e^{-\left(\frac{t}{\theta}\right)^\beta}$$

From the RBD structure:

Subsystem 1 (Parallel)

Three components in parallel: $RS1(t)=1-(1-R1) (1-R2) (1-R3)$

Subsystem 2 (Series)

Two components in series: $RS2(t)=R7 \times R8$

Subsystem 3 (Parallel + Series)

Components 4 and 5 in parallel, then in series with 6:

$$R45(t)=1-(1-R4) (1-R5)$$

$$RS3(t)=R45(t) \times R6$$

Subsystem 1 is in series with a parallel combination of Subsystems 2 and 3:

$$R_{sys}(t)=RS1(t)[1-(1-RS2(t)) (1-RS3(t))]$$

The above expressions are evaluated at:
 $t=100, 500, 1000$ hours

Using:

- Exponential reliability for Subsystems 1 & 2
- Weibull reliability for Subsystem 3

The computed results give:

- ✓ Individual component reliabilities
- ✓ Subsystem reliabilities
- ✓ Final system reliability

Result of Analytical Experiment:

$$\begin{aligned} R1(100)=R2(100)=R3(100) &= 0.9048 \\ R4(100)=R5(100)=R6(100) &= 0.9689 \\ R7(100)=R8(100) &= 0.8869 \\ \text{Analytical System Reliability}(100) &= 0.9923 \end{aligned}$$

$$\begin{aligned} R1(500)=R2(500)=R3(500) &= 0.6065 \\ R4(500)=R5(500)=R6(500) &= 0.7022 \\ R7(500)=R8(500) &= 0.5488 \\ \text{Analytical System Reliability}(500) &= 0.7028 \end{aligned}$$

$$\begin{aligned} R1(1000)=R2(1000)=R3(1000) &= 0.3679 \\ R4(1000)=R5(1000)=R6(1000) &= 0.3679 \\ R7(1000)=R8(1000) &= 0.3012 \\ \text{Analytical System Reliability}(1000) &= 0.2179 \end{aligned}$$

Fig.6.1.1: Analytical values of Reliability

6.2 Algorithm:

(For $N = 100000$ trials at $t = 100, 500, 1000$ hours)

Step 1: Start

Step 2: Initialize parameters.

Initialize all counters for:

- System reliability at $t = 100, 500, 1000$
- Subsystem 1, 2, 3 reliabilities at $t = 100, 500, 1000$

Set all counters to zero.

Step 3: Repeat for $i = 1$ to N

3.1 Generate random numbers: Generate eight uniformly distributed random numbers

3.2 Evaluate component survival at $t = 100$ hours Compare random numbers with analytical reliabilities

3.3 Evaluate subsystems:

Subsystem 1 works if comp1 OR comp2 OR comp3

Subsystem 2 works if comp7 AND comp8

Subsystem 3 works if (comp4 OR comp5) AND comp6

3.4 Evaluate system: System works if Subsystem1 AND (Subsystem2 OR Subsystem3)

3.5 Update counters for $t = 100$ Increment system and subsystem success counters.

3.6 Repeat Steps 3.2–3.5 for:

- $t = 500$ hours
- $t = 1000$ hours

(using corresponding analytical reliabilities)

Step 4: Compute experimental reliabilities

For each time:

$R_{\text{system}}(t) = \text{successful_system_trials} / N$

$R_{\text{subsystem}}(t) = \text{successful_subsystem_trials} / N$

Step 5: Display results

Print system and subsystem reliabilities for:

- 100 hours
- 500 hours
- 1000 hours

Step 6: End

```

System Experimental Reliability (100) = 0.9924
subSystem1 Experimental Reliability (100) = 0.9991
subSystem2 Experimental Reliability (100) = 0.7876
subSystem3 Experimental Reliability (100) = 0.9684

```

```

System Experimental Reliability (500) = 0.7012
subSystem1 Experimental Reliability (500) = 0.9397
subSystem2 Experimental Reliability (500) = 0.2997
subSystem3 Experimental Reliability (500) = 0.6391

```

```

System Experimental Reliability (1000) = 0.2150
subSystem1 Experimental Reliability (1000) = 0.7462
subSystem2 Experimental Reliability (1000) = 0.0912
subSystem3 Experimental Reliability (1000) = 0.2193

```

Fig 6.2.1. Simulation values of Reliability

(c) Determination of System Lifetime Distribution

To determine the lifetime distribution of the system, Monte Carlo simulation is employed to generate a large number of system time-to-failure (TTF) samples based on the statistical failure characteristics of individual components. Exponentially distributed random lifetimes are generated for components in Subsystems 1 and 2 using the inverse transform method, while Weibull-distributed lifetimes are generated for Subsystem 3 components. The system failure time for each trial is obtained by applying the reliability block diagram logic to the simulated component lifetimes. The collection of simulated system failure times is then represented using a histogram to visualize the probability density and overall shape of the system lifetime distribution.

(d) Lifetime Distribution Modelling Using Weibull and Normal Models

To model the statistical behaviour of the system lifetime, the simulated failure times are analysed using probability plotting techniques. The Weibull lifetime model is applied due to its flexibility in representing aging and wear-out behaviour in reliability systems. In addition, a Normal distribution model is evaluated for comparison purposes. The unknown distribution parameters are estimated using the probability plot method, which assesses the linearity of transformed failure data. The goodness of fit of each model is visually examined to identify the most appropriate lifetime distribution for the system.

6.3 Algorithm:

Step 1: Start

Step 2: Define distribution parameters $\lambda_1 = 0.001$, $\lambda_2 = 0.0012$, $\beta = 1.5$, $\theta = 1000$,

$N = 100000$

Step 3: Initialize array to store system failure times

Step 4: Repeat for $j = 1$ to N

4.1 Generate component lifetimes

For exponential components: $t = -(1/\lambda) \ln(1 - F(t))$

For Weibull components: $t = \theta [-\ln(1 - F(t))]^{1/\beta}$

4.2 Compute subsystem lifetimes

Subsystem 1 lifetime = $\max(t_1, t_2, t_3)$

Subsystem 3 lifetime = $\min(\max(t_4, t_5), t_6)$

Subsystem 2 lifetime = $\min(t_7, t_8)$

4.3 Compute system lifetime

$T_{sys} = \min(\text{Subsystem1}, \max(\text{Subsystem3}, \text{Subsystem2}))$

4.4 Store T_{sys} in array

Step 5: Plot histogram of T_{sys}

Step 6: Apply probability plots

- Fit Weibull distribution
- Fit Normal distribution

Step 7: Estimate parameters from plots

Step 8: End

Result:

System's TTF = 734.9000

Mean system TTF = 734.90

Median system TTF = 681.31

6.4 Result and outcome:

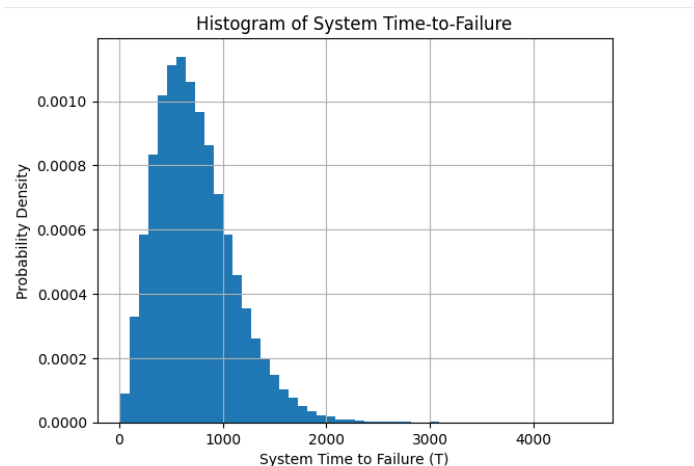
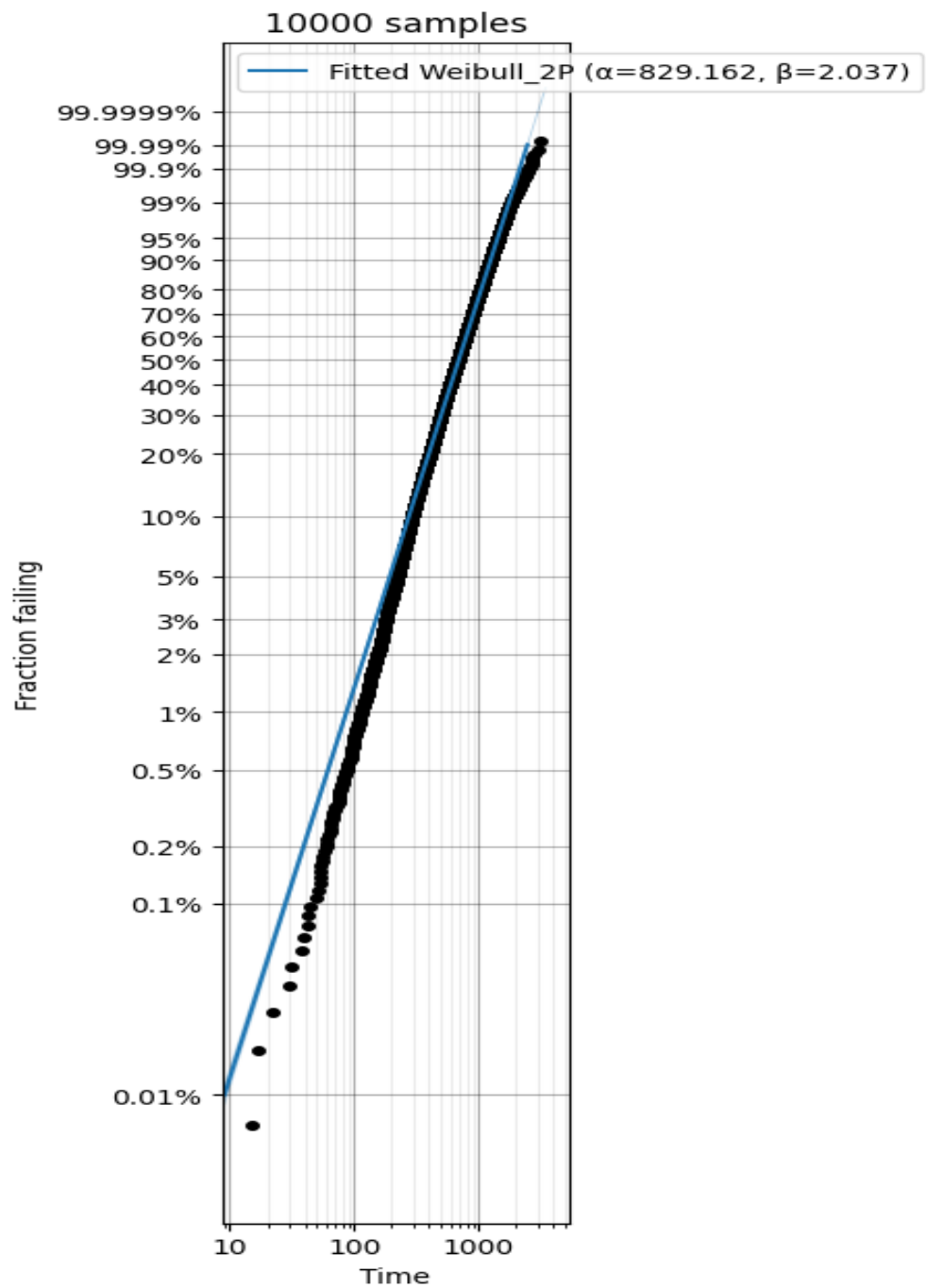
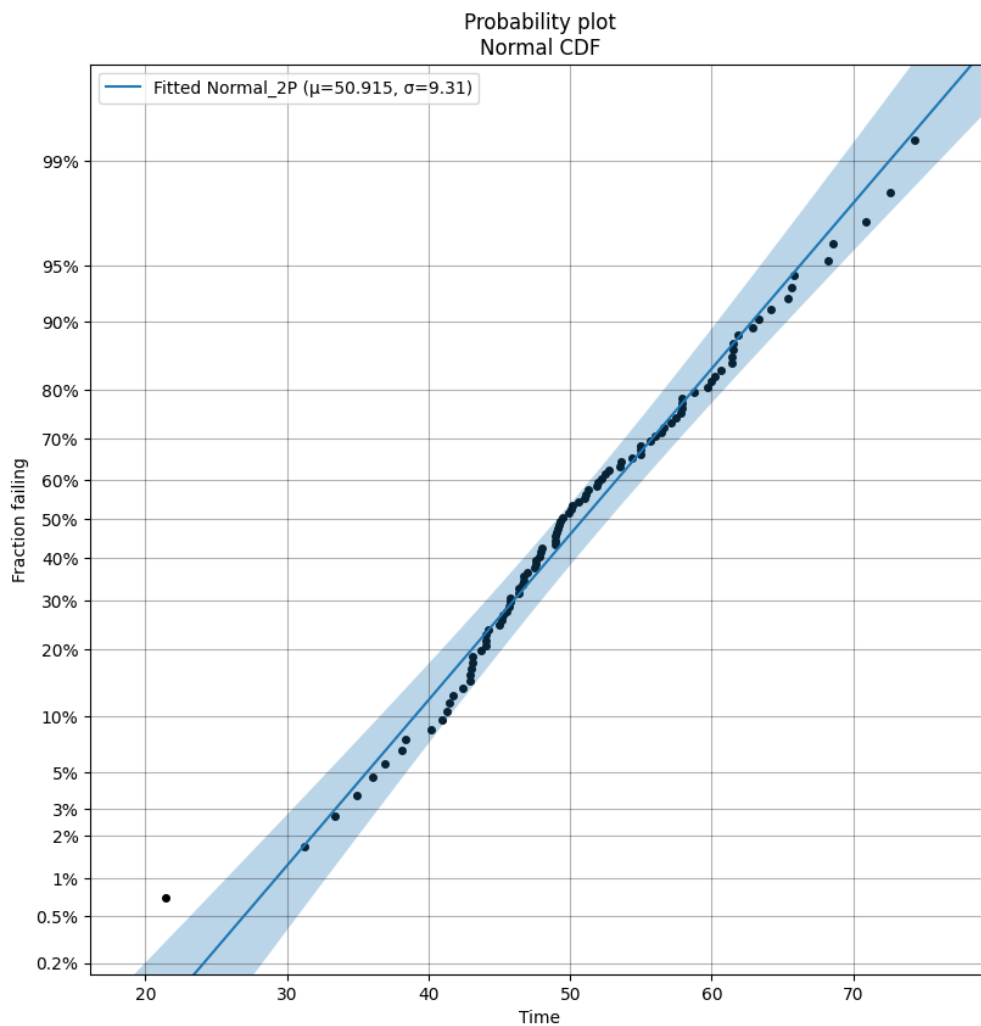


Fig 6.4.1. Histogram of system TTF



$$\theta = 829.162hr, \beta = 2.037$$

Fig.6.4.2. Weibull Probability Plot of TTF values



$$\mu = 50.915hr, \sigma = 9.31hr$$

Fig 6.4.3: Normal Distribution probability plot of TTF values

6.5 Observations:

1. The Monte Carlo simulation produced a large set of system time-to-failure (TTF) values representing realistic system behaviour over time.
2. The histogram of system TTF shows a **right-skewed distribution**, where most failures occur around a central time range while a small number of systems survive much longer.
3. The mean system lifetime is higher than the median, confirming the skewed nature of the distribution.
4. The Weibull probability plot exhibits an approximately **linear trend**, indicating a strong agreement between the simulated data and the Weibull lifetime model.
5. The Normal probability plot shows noticeable deviations, especially in the tail regions, suggesting a poor fit for symmetric lifetime modelling.

6. The spread of failure times increases as time progresses, reflecting progressive system aging.

6.6 Discussion:

The lifetime distribution obtained from Monte Carlo simulation captures the combined effects of exponential and Weibull component behaviours within the system.

Subsystems governed by exponential distributions show constant failure rates, while the Weibull-based subsystem introduces time-dependent aging effects. When combined through the reliability block diagram, the overall system lifetime exhibits non-symmetric behaviour typical of real engineering systems.

The right-skewed histogram confirms that failures are not evenly distributed but follow a wear-out pattern.

The Weibull probability plot's strong linearity demonstrates that the Weibull model effectively represents this aging behaviour, making it appropriate for reliability prediction and maintenance planning.

In contrast, the Normal model assumes symmetry and constant variability, which does not match physical failure mechanisms, leading to poorer fit.

The Monte Carlo method successfully reproduces realistic lifetime patterns that align with analytical reliability theory.

6.7 Conclusion from this experiment:

The Monte Carlo simulation successfully generated realistic system failure times and revealed the system lifetime distribution characteristics.

- The system lifetime is right-skewed
- Weibull distribution provides the best lifetime model
- Normal distribution is unsuitable for failure data
- The system exhibits aging and wear-out behaviour

Overall, Weibull modelling is the most appropriate approach for predicting system reliability and failure trends, while Monte Carlo simulation proves to be an effective tool for complex lifetime analysis.

Problem _7: CAPACITY DEMAND

A mechanical system is subjected to a random applied stress **D** and possesses a random strength **C**.

Both variables follow normal distributions:

- Strength: $C \sim N(1000, 150^2)$
- Stress: $D \sim N(800, 100^2)$

(a) Derive the analytical expression for reliability: $R=P(C>D)$ and compute its value.

(b) Using Monte Carlo simulation, estimate

the reliability for:

$N=100, 1000, 10000, 100000$

(c) Calculate the percentage error for each trial size.

7.1 Methodology:

Analytical observation-

In stress–strength interference theory, a system is considered safe if the **random strength (C)**

of the component exceeds the **random applied stress (D)**. Hence, the system reliability is defined as: $R=P(C>D)$

The strength and stress are both assumed to follow **normal distributions**:

$$C \sim N(\mu_c, \sigma_c^2), D \sim N(\mu_d, \sigma_d^2)$$

Where:

$$\mu_C = 1000, \sigma^2 = 22500$$

$$\mu_D = 800, \sigma^2 = 10000$$

Distribution of the Difference Variable

Define a new random variable:

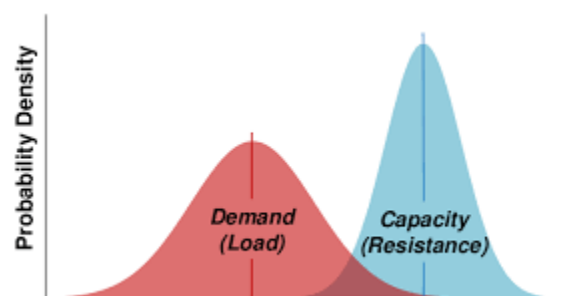
$$Z = C - D$$

Since the difference of two independent normal variables is also normally distributed:

$$Z \sim N(\mu_Z, \sigma_Z^2)$$

Where:

$$\mu_Z = \mu_C - \mu_D, \sigma_Z^2 = \sigma_C^2 + \sigma_D^2$$



$$\mu_Z = 1000 - 800 = 200, \sigma_Z^2 = 22500 + 10000 = 32500$$

$$\sigma_Z = \sqrt{32500} \approx 180.28$$

Reliability is: $R = P(Z > 0)$

$$R = \Phi(\mu_Z / \sigma_Z)$$

Where $\Phi(\cdot)$ is the standard normal cumulative distribution function.

Final Analytical Reliability

$$R = \Phi(200 / 180.28), R \approx \Phi(1.109), R \approx 0.866$$

Simulation Experiment: Monte Carlo simulation is an effective and reliable method for estimating probabilistic system performance and converges strongly toward analytical results as sample size increases.

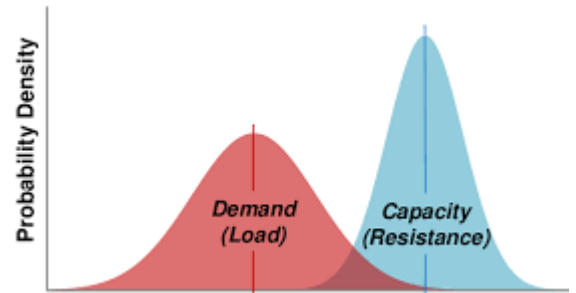
$$R = [P(D < C)]$$

$$= \int_0^C \int_0^\infty \{f(D, C) dC\} dD$$

$$= \int_0^\infty \int_0^\infty I[D < C] f(D, C) dC dD$$

$$= E_f[I(D < C)]$$

$$= \frac{1}{N} \sum_{i=1}^N I[(d_i < c_i)] \quad \text{where } d_i, c_i \sim f(D, C)$$



7.2 Algorithm:

Step 1: Start

Step 2: Define Input Parameters

1. Set mean of strength variable
→ $\mu_{euc} = 1000$
2. Set mean of stress variable
→ $\mu_{eud} = 800$
3. Set variances:
→ $\sigma_c^2 = 22500$
→ $\sigma_d^2 = 10000$

4. Compute standard deviations:
 $\rightarrow \sigma_c = \sqrt{22500}$
 $\rightarrow \sigma_d = \sqrt{10000}$
5. Compute combined mean and standard deviation:
6. $\mu = \mu_{euc} - \mu_{eud}$
7. $\sigma = \sqrt{(\sigma_c^2 + \sigma_d^2)}$

Step 3: Compute Analytical Reliability

1. Calculate Z-value:
2. $z = \mu / \sigma$
3. Find analytical reliability using normal CDF:
4. $Rel = \Phi(z)$
5. Display analytical reliability

Step 4: Define Trial Sizes

Step 5: Monte Carlo Simulation Loop

For each N in trial_list:

Set:

success = 0

Repeat N times:

a. Generate random strength value:

$c \sim \text{Normal}(\mu_{euc}, \sigma_c)$

b. Generate random stress value:

$d \sim \text{Normal}(\mu_{eud}, \sigma_d)$

c. If: $c > d$

then:

success = success + 1

Compute experimental reliability:

experimental = success / N

Compute percentage error:

percent_error = $|(\text{experimental} - Rel) / Rel| \times 100$

Store results in lists

Print results

Step 6: Plot the graph

Step 7: End

7.3 Result:

Analytical Reliability = 0.8663712534228061

TRIALS	EXPERIMENTAL	PERCENT ERROR (%)
100	0.920000	6.1900
1000	0.859000	0.8508
10000	0.869600	0.3727
100000	0.866510	0.0160

Fig 7.3.1: Table of simulation result

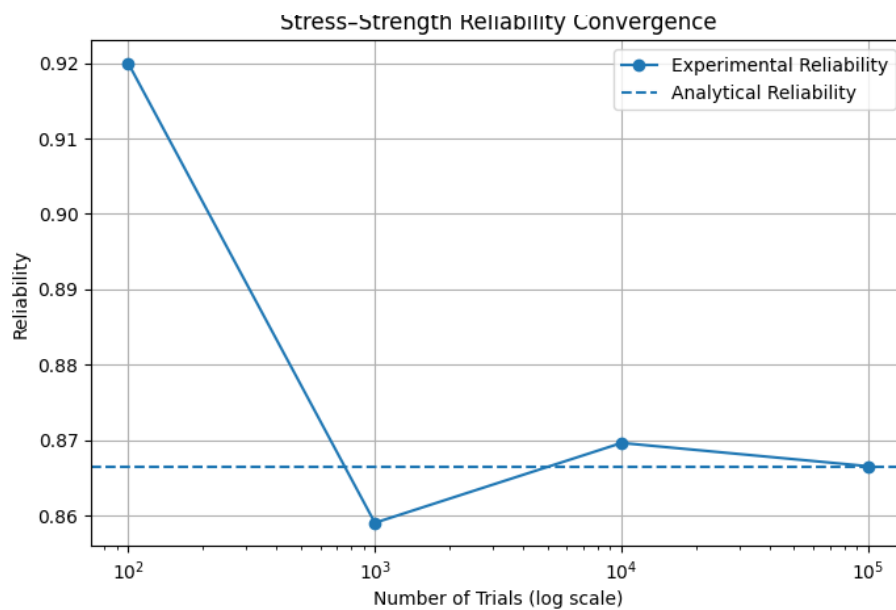


Fig 7.3.2: Graph of Experimental Reliability vs Analytical Reliability

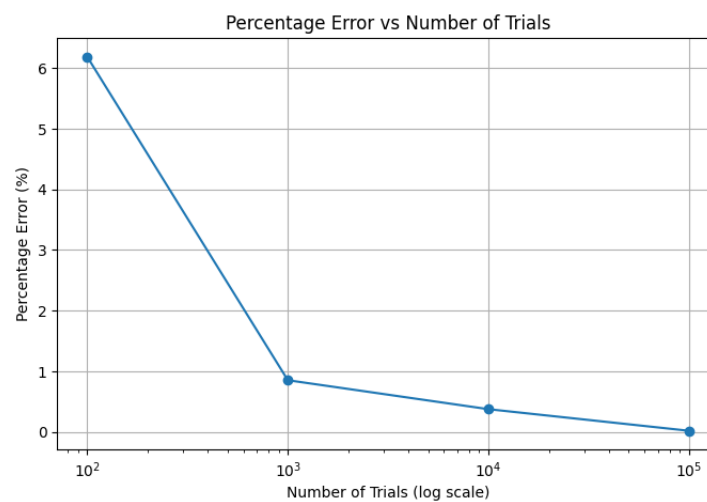


Fig.7.3.3: Graph of Percentage error in Reliability vs Number of Trials

7.4 Observation:

1.From the reliability convergence plot:

- At low trials (100), results fluctuate significantly
- As the number of trials increases, experimental reliability approaches the analytical value

2.From the percentage error plot:

- Error decreases rapidly with increasing number of simulations
- Nearly zero error is achieved at 100,000 trials

7.5. Discussion:

- The Monte Carlo simulation initially shows high variability due to insufficient random samples (law of small numbers).
- With larger sample sizes, random fluctuations reduce, and the estimated probability converges toward the true analytical reliability.
- This behaviour demonstrates the **Law of Large Numbers**, where the experimental mean approaches the theoretical expectation.
- The logarithmic scale highlights how convergence improves exponentially with more trials.
- The close agreement at 100,000 trials confirms the correctness of both:
 - The analytical formulation using normal distributions
 - The numerical simulation approach

7.6. Conclusion:

- The stress–strength reliability of the system is approximately : $R \approx 0.866$
- Monte Carlo simulation successfully validates the analytical solution.
- Increasing the number of trials significantly improves accuracy.
- For practical engineering applications:
 - Few thousand trials provide good estimates
 - Large trials ensure near-exact reliability

Problem_8: Mean Time To Failure (MTTF) calculation

8.1 METHODOLOGY:

➤ Analytical Explanation-

In reliability engineering, the Mean Time To Failure (MTTF) represents the expected lifetime of a system

$MTTF = \int_0^{\infty} R(t) dt$, where $R(t)$ is the reliability function.

MTTF of Weibull Distribution $= \theta \tau (1 + \frac{1}{\beta}) = 5000 \times \tau (1 + \frac{1}{2}) = 4431.13 \text{ hr}$,

where $\theta = 5000 \text{ hr}$, $\beta = 2$

➤ Simulational Explanation-

The Mean Time To Failure (MTTF) in reliability engineering is defined as the total expected operating time of a system before failure. It is given by the integral of the reliability function:

$$MTTF = \int_0^{\infty} R(t) dt$$

where $R(t)$ is the probability that the system is still functioning at time t .

Reliability Model (Weibull Distribution)

In this case, the reliability function is assumed to follow a Weibull form:

$$R(t) = \exp [- (t / \theta)^{\beta}]$$

where:

θ = scale parameter

β = shape parameter

So the MTTF becomes: $MTTF = \int_0^{\infty} \exp [- (t / \theta)^{\beta}] dt$

Since integrating to infinity is not practical for numerical methods, the upper limit is replaced by a sufficiently large value $t_{\max}$:

$$MTTF \approx \int_0^{t_{\max}} \exp [- (t / \theta)^{\beta}] dt$$

$t_{\max}$ is chosen such that $R(t_{\max}) \approx 0$.

Variable Transformation

To apply Monte Carlo integration, the variable is normalized:

$$x = t / t_{\max}$$

$$t = x \cdot t_{\max}$$

$$dt = t_{\max} dx$$

Substituting:

$$MTTF \approx t_{\max} \int_0^1 \exp [- (x t_{\max} / \theta)^\beta] dx$$

Monte Carlo Estimation

Let x_i be uniformly distributed random numbers in $[0,1]$. Then:

$$MTTF \approx (t_{\max} / N) \sum \exp [- (x_i t_{\max} / \theta)^\beta]$$

for $i = 1$ to N

5.2. Algorithm:

STEP 1: Initialize sum = 0

STEP 2: FOR $i = 1$ to N DO

2.1 Generate a random number x uniformly in the range $[0,1]$

2.2 Compute reliability value:
 $y = \exp [- (x \times t_{\max}) / \theta)^\beta]$

2.3 Update sum:
sum = sum + y

STEP 3: Compute Monte Carlo MTTF:

$$mttf_mc = (t_{\max} / N) \times \text{sum}$$

STEP 4: Compute percentage error:

$$\text{error} = | (mttf_mc - mttf_exact) / mttf_exact | \times 100$$

STEP 5: Display mttf_mc and error

STEP 6: END

8.2. Result:

Analytical MTTF: 4431.13

Experimental MTTF: 4430.94

Percentage Error: 0.0045% when $t_{\max}=11700\text{hr}$

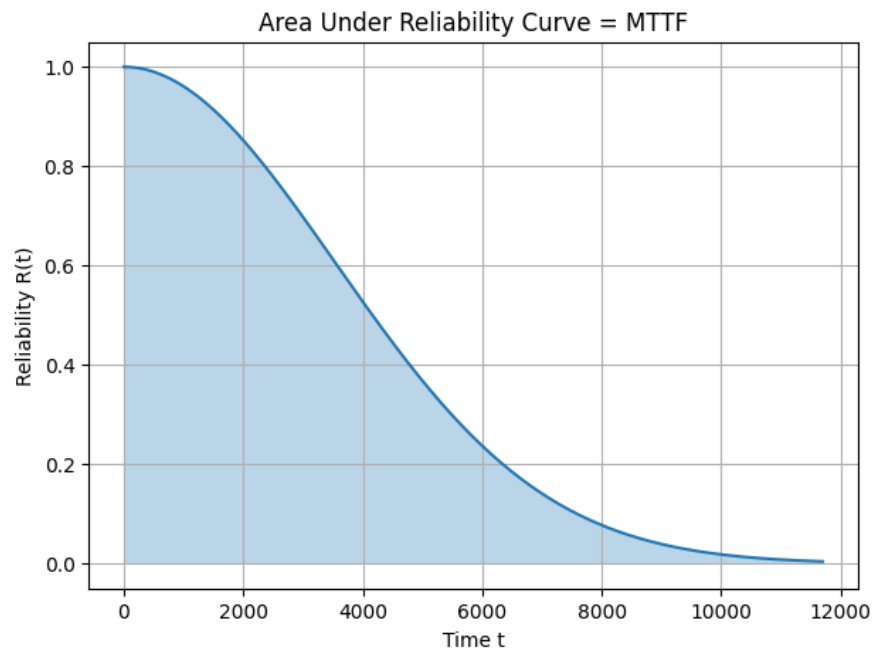


Fig 8.2.1. Graphical representation of Reliability vs Time

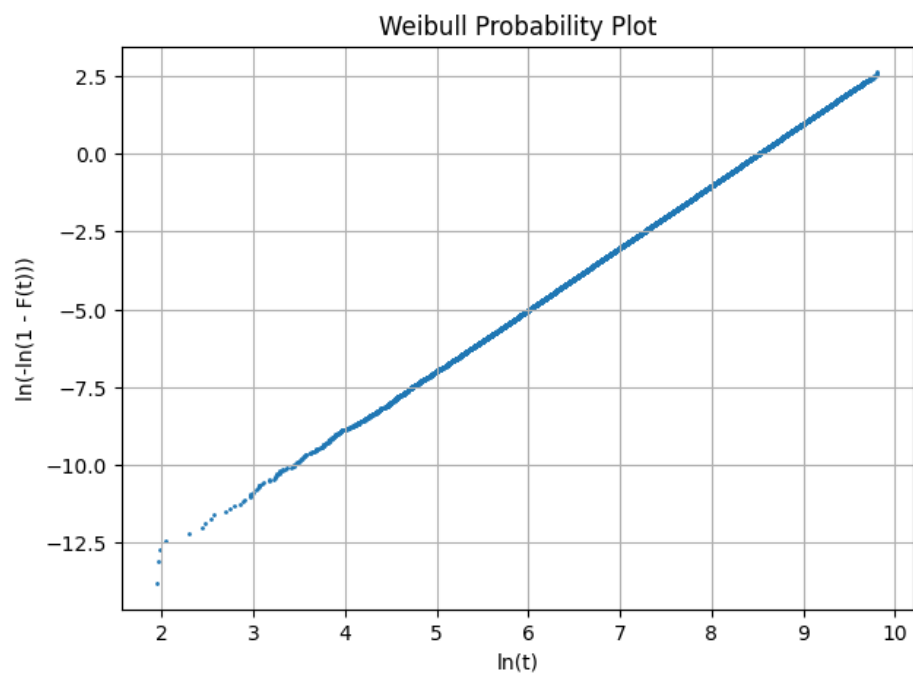


Fig 8.2.2: Weibull Probability Plot

8.3 OBSERVATIONS:

1. The Weibull probability plot shows the data points forming an approximately straight line.
2. This linear pattern confirms that the failure times follow a Weibull distribution.
3. The slope of the line is positive, indicating a shape parameter $\beta > 1$, which represents wear-out type failures.
4. The reliability curve starts at $R(0) = 1$ and decreases smoothly toward zero as time increases.
5. The shaded region under the reliability curve represents the Mean Time To Failure (MTTF).
6. The curve gradually approaches zero near $t = t_{\text{max}}$, confirming that the selected upper limit is sufficient for numerical integration.

8.4. DISCUSSION:

The Weibull probability plot forms an approximately straight line, confirming that the failure data follows a Weibull distribution and validating the reliability model used. The reliability curve shows a gradual decrease from 1 to 0, representing increasing failure probability over time. The shaded area under this curve corresponds to the Mean Time To Failure (MTTF), which is estimated numerically using Monte Carlo sampling. The close agreement between analytical and simulated MTTF values demonstrates the accuracy and effectiveness of the Monte Carlo method for reliability analysis.

The Monte Carlo method successfully estimates the Mean Time To Failure for a Weibull reliability model. The Weibull probability plot confirms that the failure data follows a Weibull distribution, validating the use of the reliability function.

The reliability curve clearly illustrates that MTTF is equal to the area under the survival curve. The numerical results closely match the analytical MTTF obtained using the Gamma function, demonstrating the accuracy of the simulation approach.

Therefore, Monte Carlo integration is a reliable and efficient technique for MTTF estimation and can be applied to complex reliability models where closed-form solutions do not exist.

Problem_9: Integration - Estimate the value of the definite integral $\int_0^1 x^3 dx$

using:

9.1 Methodology:

1. Analytical method
2. Monte Carlo simulation method

➤ ANALYTICAL METHODOLOGY

The given function is:

$$f(x) = x^3$$

The definite integral is evaluated analytically as follows:

$$\int_0^1 x^3 dx$$

$$= [x^4 / 4]_0^1$$

$$= (1^4 / 4) - (0^4 / 4)$$

$$= 1/4$$

$$= 0.25$$

Analytical Result = 0.25

➤ SIMULATION METHODOLOGY –

- Generate N random numbers uniformly between 0 and 1
- Evaluate the function $f(x) = x^3$ at each random value
- Compute the average of the function values
- Multiply the average by the interval length $(b - a) = 1$

$$\text{Integral} \approx (b - a)/N \times \sum f(x_i)$$

9.2. Algorithm:

STEP 1: Set total = 0

STEP 2: FOR i = 1 to N DO

2.1 Generate a random number x uniformly in the interval [a, b]

2.2 Evaluate the function value: $f = x^3$

2.3 Update total: total = total + f

STEP 3: Compute the Monte Carlo integral: $I = (b - a)/N \times \text{total}$

STEP 4: Display I

STEP 5: END

9.3 Result and outcome:

TRIALS (N)	$f(x)=x^3$	Monte Carlo Integral
100	0.005635	0.298066
1000	0.007393	0.239595
10000	0.205356	0.254954
100000	0.402776	0.248405

Fig 9.3.1: Table of Simulation Approach

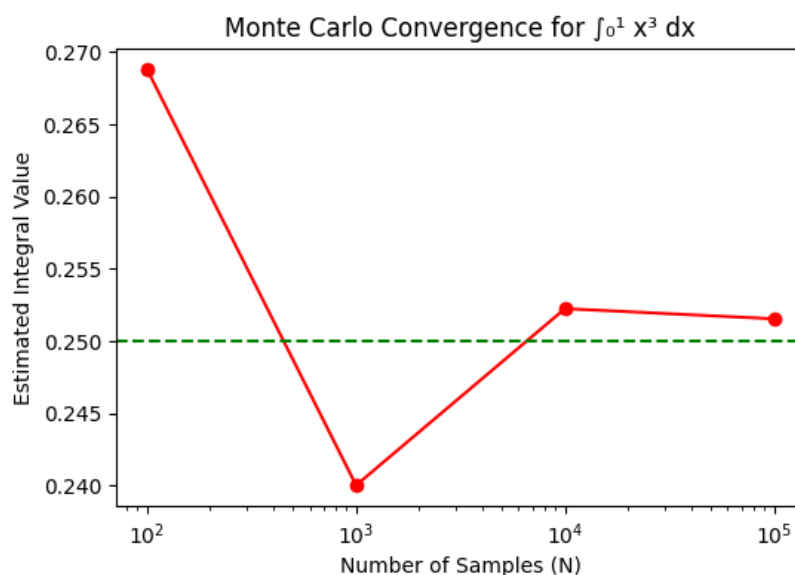


Fig 9.3.2: Graphical representation of Estimated Integral Value vs Number of Samples

9.4 **OBSERVATION:**

1. The Monte Carlo estimated values fluctuate for small sample sizes.
2. As the number of samples (N) increases, the estimated integral moves closer to the true value (0.25).
3. The plotted points gradually converge toward the horizontal dashed line representing the analytical solution.
4. The variation in results decreases with increasing N, indicating improved accuracy.

9.5 **DISCUSSION:**

The convergence plot illustrates the behaviour of the Monte Carlo integration method for estimating the integral of x^3 over the interval $[0, 1]$. For small values of N, the estimates show noticeable deviation due to random sampling effects. However, as the number of samples increases, the computed values approach the analytical solution of 0.25. This demonstrates the law of large numbers, where increasing random samples reduces statistical error. The horizontal true-value line provides a clear reference showing how the numerical results converge toward the exact integral value.

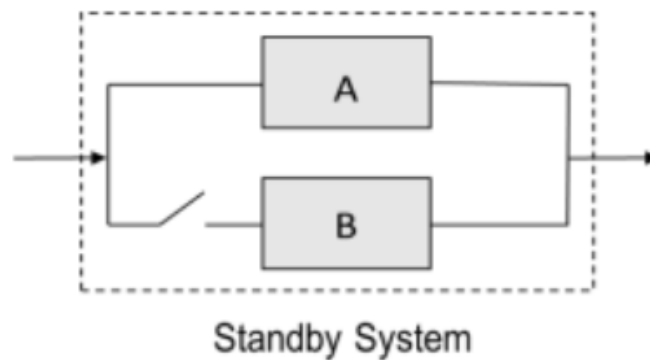
The Monte Carlo method effectively approximates the definite integral $\int_0^1 x^3 dx$. Although the results fluctuate for smaller sample sizes, increasing the number of trials significantly improves accuracy. The convergence toward the analytical value confirms the reliability of Monte Carlo simulation for numerical integration, especially for problems where analytical solutions may be difficult or unavailable.

Problem_10: Q1: Reliability of a two-component standby system

The reliability block diagram (RBD) of a two-component standby system with switching is shown in the figure below. The TTF of Component A follows exponential distribution with $\lambda = 0.001$ failures/hr, and the TTF of Component B follows Weibull distribution with shape parameter (β) = 1 and scale parameter (θ) = 1000 hr.

(a) Assuming perfect switching, determine the system reliability after 100, 500, and 1000 hours of operation using Monte Carlo simulation and verify your results using analytical calculations.

(b) Now, assuming imperfect switching with probability of 0.95, determine the system reliability after 100, 500, and 1000 hours of operation using Monte Carlo simulation and verify your results using analytical calculations. Which one is more reliable?



10.1. Methodology:

➤ Analytical approach-

(a) Phase 1: Component A operates : System survives if A does not fail: $R_A(t) = e^{-\lambda t}$

Phase 2: A fails at time τ , then B takes over

-Probability A fails in small interval $d\tau$: $f_A(\tau)d\tau = \lambda e^{-\lambda\tau}d\tau$

-After switching, B must survive remaining time $(t-\tau)$: $R_B(t-\tau) = e^{-(t-\tau)^\beta} dt$

Combine both effects:

$R_{sys}(t)$ = no switching P (A survives entire time) + with switching P (A fails at τ AND B survives)

$$R_{sys}(t) = e^{-\lambda t} + \int_0^t \lambda e^{-\lambda\tau} e^{-(t-\tau)^\beta} dt$$

Intuitive Meaning of the Result: when $\beta = 1$,

$$R_{sys}(t) = e^{-\lambda t}(1 + \lambda t)$$

(b) Imperfect Switching ($p = 0.95$) – Now not every failure of A leads to successful backup. So only a fraction p of the rescue cases work:

$$R_{sys}(t) = e^{-\lambda t}(1+p \lambda t)$$

➤ Simulation Approach –

Step 1: Generate lifetimes using inverse transform method -

For each trial:

- Sample T_a (exponential)
- Sample T_b (Weibull)

Step 2: Apply switching logic-

- **Perfect switching ($p = 1$):** $T_{sys} = T_a + T_b$
- **Imperfect switching ($p < 1$):**
 - Generate $U \sim U(0,1)$
 - If $U \leq p$: $T_{sys} = T_a + T_b$
 - Else: $T_{sys} = T_a$

Step 3: Check survival- For mission time t : If $T_{sys} > t \Rightarrow$ system survives

Step 4: Estimate reliability- $R(t) = \text{Number of survivals} / N$

10.2. Algorithm:

Step 1: Initialize parameters

- $\lambda = 0.001$, $\theta = 1000$
- β (use 1 depending on case)
- Switching probability:
 - $p = 1 \rightarrow$ Perfect
 - $p = 0.95 \rightarrow$ Imperfect
- Define trial sizes N

Step 2: For each N : Repeat simulation N times

Step 3: Generate random samples: $F_a, F_b \sim U(0,1)$

Step 4: Convert to lifetimes

- Exponential (a): $T_a = \frac{-\ln(1-F_a)}{\lambda}$
- Weibull (b): $T_b = \theta(-\ln(1-F_b))^{(1/\beta)}$

Step 5: Apply switching logic

- **Perfect switching ($p = 1$):** $T_{sys} = T_a + T_b$
- **Imperfect switching ($p < 1$):**
 - Generate $U \sim U(0,1)$

- If $U \leq p$:

$$T_{\text{sys}} = T_a + T_b$$

- Else: $T_{\text{sys}} = T_a$

Step 6: Check survival - For mission time t (100, 500, 1000) hours:

If $T_{\text{sys}} > t \Rightarrow$ system survives

Step 7: Estimate reliability – $R(t) = \text{success count} / N$

Step 8: Store & plot results - Store reliability values, Plot vs N (log scale for convergence)

Step 9: Stop

10.3. Results and outcomes:

a) Perfect switching –

Analytical Reliability at $t(100)$ is = 0.9953211598395556
 Analytical Reliability at $t(500)$ is = 0.9097959895689501
 Analytical Reliability at $t(1000)$ is = 0.7357588823428847

Fig: 10.3.1: The analytical reliability of perfect switching standby system

TRIALS(N)	R(100)	R(500)	R(1000)
100	1.0000	0.9500	0.7900
1000	0.9920	0.8960	0.7250
10000	0.9951	0.9067	0.7315
100000	0.9955	0.9107	0.7332
1000000	0.9953	0.9101	0.7358

Fig: 10.3.2: Experimental reliability of perfect switching standby system

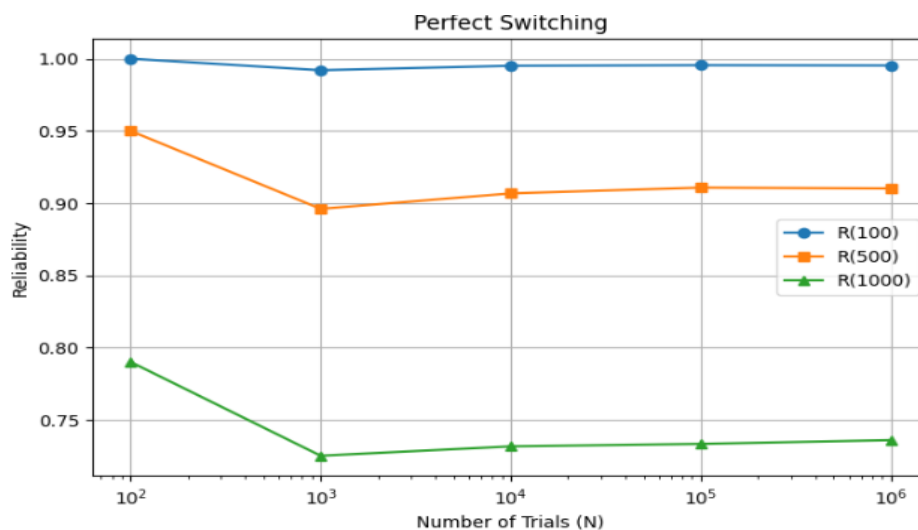


Fig 10.3.3: Graph plot of perfect switching standby system

b) Imperfect switching –

Analytical Reliability at $t(100)$ is = 0.9907969727493756
 Analytical Reliability at $t(500)$ is = 0.8946327230761344
 Analytical Reliability at $t(1000)$ is = 0.7173649102843125

Fig: 10.3.4: The analytical reliability of imperfect switching standby system

TRIALS(N)	R(100)	R(500)	R(1000)
100	1.0000	0.9000	0.6800
1000	0.9910	0.8980	0.7300
10000	0.9904	0.8931	0.7102
100000	0.9909	0.8944	0.7194
1000000	0.9909	0.8947	0.7176

Fig: 10.3.5: Experimental reliability of imperfect switching standby system

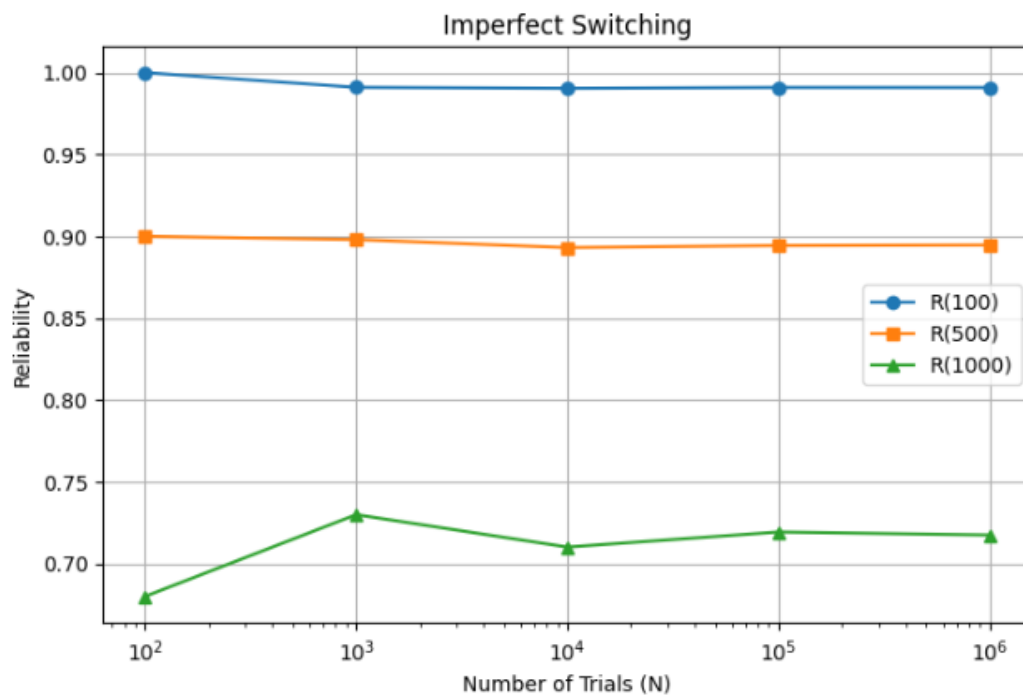


Fig 10.3.6: Graph plot of perfect switching standby system

10.4. Observations from Monte Carlo Simulation of Standby System:

1. **Convergence Behavior**

As the number of trials (N) increases, the estimated reliability values stabilize and converge to constant values. This confirms the accuracy of the Monte Carlo method.

2. **Effect of Standby Operation**

The system reliability is higher than that of a single component because the standby component (B) extends the total system lifetime.

3. **Perfect vs Imperfect Switching**

- Perfect switching ($p = 1$) gives higher reliability.
- Imperfect switching ($p = 0.95$) reduces reliability due to possible switching failure.

4. **Impact of Time**

Reliability decreases as mission time increases (100 → 500 → 1000 hours), since the probability of failure increases over time.

5. **Effect of Switching Probability**

Lower switching probability reduces the contribution of the standby component, making the system less reliable.

6. **Role of Weibull Parameter (β)**

- For ($\beta = 1$): constant failure rate (exponential case).
- For ($\beta > 1$): increasing failure rate, leading to faster degradation of reliability.

7. **Accuracy of Simulation**

The simulation results closely match analytical results, validating the model and implementation.

10.5. Discussion:

The standby system significantly improves reliability compared to a single-component system. Perfect switching provides the highest reliability, while imperfect switching reduces performance due to possible switching failure. The results show that reliability decreases with time and that Monte Carlo simulation effectively estimates system reliability with good accuracy and convergence.

Problem_11: Q2: Reliability analysis of a two-component load-sharing system Consider a subsystem consisting of two identical components operating under a load-sharing configuration. The subsystem functions as long as at least one component remains operational. Each component has an exponentially distributed lifetime. When both components are operating and sharing the load equally, the failure rate of each component is $\lambda_L = 0.001$ failures/hour. Upon failure of one component, the entire load is transferred to the remaining component, causing its failure rate to increase to $\lambda_H = 0.003$ failures/hour. The subsystem is considered failed when both components have failed.

Using Monte Carlo simulation, estimate the reliability of the subsystem for mission times of interest and obtain the reliability profile. Derive the analytical expression for the subsystem reliability and verify the simulation results.

11.1 Methodology:

➤ Analytical approach-

1. System Description

- Two **identical components**
- System works if **at least one survives**
- Failure rates:
 - When both are active: $\lambda_L = 0.001$ failures/hour
 - When one fails (remaining carries full load): $\lambda_H = 0.003$ failures/hour

2. Modelling Approach



State	Description	Transition
2	Both components working	→ State 1
1	One failed, one working	→ State 0
0	Both failed (system failure)	Absorbing

Table 11.1.1: State transition diagram of the system

3. Transition Rates

- From state 2 → 1:
Since both components can fail independently: $2\lambda_L = 2(0.001)$
- From state 1 → 0: $\lambda_H = 0.003$

4. Differential Equations

$$R(t) = e^{-2\lambda t} + \int_0^t \lambda e^{-2\lambda t} e^{-\lambda(t-t_1)} dt_1 + \int_0^t \lambda e^{-2\lambda t} e^{-\lambda(t-t_2)} dt_2$$

Simplified:

$$R(t) = 3e^{-0.002t} - 2e^{-0.003t}$$

- Simulation Approach: Simulate many system lifetimes

For each trial:

Generate **time to first failure** (rate 2λ)

Generate **time to second failure** (rate λH)

Total life = $T = T_1 + T_2$

Estimate reliability:

$$R(t) = (\text{Number of trials with } T > t) / N$$

11.2. Algorithm:

Step 1: Start

Step 2: Generate random failures

$F_a = \text{random.uniform}(0,1)$, $F_b = \text{random.uniform}(0,1)$

these are random numbers used to simulate uncertainty.

Step 3: Time to first failure

$t_a = -\text{math.log}(1-F_a)/(2*\text{lamda})$

Step 4: Time to second failure

$t_b = -\text{math.log}(1-F_b)/(\text{lamda} \star)$

Step 5: Total system failure time : $t_{\text{total}} = t_a + t_b$

Step 6: Reliability Estimation

if $t_{\text{total}} > 100 \rightarrow$ system survived 100 hrs

if $t_{\text{total}} > 1000 \rightarrow$ system survived 1000 hrs

Then estimate probability:

$$R(t) = (\text{Number of times system survives beyond } t) / N$$

Step 7: Repeating for Accuracy

Repeat this method for different trial sizes: trial_list = [100, 1000, 10000, ...]

- ☐ Larger N → more accurate results
- ☐ Shows **convergence toward analytical value**

Step 8: Stop

11.3. Result and outcomes:

Analytical Reliability for 100 hr is = 0.9745558178705098
Analytical Reliability for 1000 hr is = 0.3064317129741102

Fig 11.3.1: The analytical reliability of load sharing system

TRIALS(N)	Reliability_100	Reliability_1000
100	0.9500	0.2700
1000	0.9760	0.2880
10000	0.9740	0.3087
100000	0.9749	0.3076
1000000	0.9744	0.3078

Fig 11.3.2: The experimental reliability of load sharing system

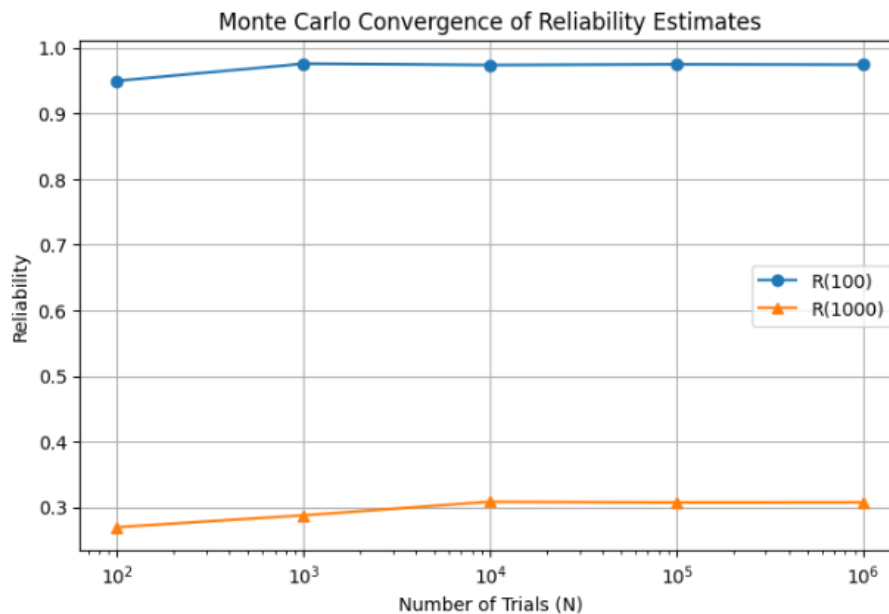


Fig 11.3.3: Graph plot of load sharing system

11.4. Observations:

- **Convergence:**
As the number of trials N increases, the estimated reliability values become stable and converge to the analytical result.
- **Accuracy improves with N :**
Small N → fluctuating results
Large N → smooth and accurate estimates
- **Reliability decreases with time:**
 $R(100) > R(1000)$ because longer mission time → higher chance of failure
- **Load-sharing effect visible:**
Reliability drops faster after first failure due to increased failure rate λH
- **Simulation matches theory:** Our simulation values match with the analytical values

11.5 Discussion:

The **load-sharing system reliability decreases over time** due to increased stress on the remaining component after the first failure. The **Monte Carlo simulation successfully estimates reliability** by mimicking real failure behavior using random sampling. As the number of trials increases, the simulation **converges to the analytical result**, confirming its correctness.

Problem_12: Q3: Reliability analysis of a KooN system Consider a k-out-of-n (koon) system, where the system performance depends on the operational status of n components, and the system functions according to either a K-out-of-N:G or K-out-of-N:F criterion. You may choose any suitable values of k and n such that $1 \leq k \leq n$.

(a) Assume that all components are identical, statistically independent, and have exponentially distributed lifetimes. You may choose an appropriate failure rate for the components. Using Monte Carlo simulation, evaluate the reliability of the system for selected mission times and verify the simulation results analytically using the closed-form reliability expression for a k-out-of-n system.

(b) Extend the analysis to a heterogeneous system, in which the component lifetimes remain statistically independent but are non-identical, with different failure rates (to be chosen appropriately).

12.1 Methodology(a) :

➤ Analytical Approach:

System Description

A **k-out-of-n:G system** is considered, where the system functions if at least **k out of n components are operational** at a given mission time t.

In this study:

- Number of components: $n=4$
- Minimum required functioning components: $k=2$
- System type: **2-out-of-4:G system**

Each component is assumed to fail independently

Assumptions

- All components are **identical**
- Lifetimes follow an **exponential distribution**
- Failure rate: $\lambda=0.005$
- Mission time: $t=200$

System Reliability Expression

For a k-out-of-n system with identical components:

$$R_{sys}(t) = \sum_{i=k}^n n_{C_i} p^i (1-p)^{(n-i)}$$

Where:

- $p=e^{-\lambda t}$

➤ Simulation approach:

For each trial:

1. Generate 4 random numbers U_1, U_2, U_3, U_4
2. Convert them into lifetimes: $T_i = -\frac{\ln(1-U)}{\lambda}$
3. Check which components survive: $T_i \geq t$
4. Count working components: $\text{working} = \sum(T_i \geq t)$
5. System condition:

If $\text{working} \geq k \rightarrow \text{system success}$

Else $\rightarrow \text{system failure}$

12.2 Algorithm:

Step 1 : import libraries

Step 2: Initialize Storage

1. Create empty lists:
 - $N_list \rightarrow$ stores number of trials
 - $Rel_200_list \rightarrow$ stores simulated reliability

Step 3: Monte Carlo Simulation

For each trial size NNN in $trial_list$:

3.1 Initialize counter

- Set $system_success = 0$

3.2 Repeat simulation for N trials

- Generate 4 random numbers U_1, U_2, U_3, U_4
- Convert them into lifetimes: $T_i = -\frac{\ln(1-U)}{\lambda}$
- Check which components survive: $T_i \geq t$
- Count working components: $\text{working} = \sum(T_i \geq t)$
- System condition:

If $\text{working} \geq k \rightarrow \text{system success}$

Else $\rightarrow \text{system failure}$

3.3 Reliability Estimation

After N trials: $R_{sim} = (\text{Number of successful trials}) / N$

3.4 Store Results

- Append NNN to N_list
- Append R_{sim} to Rel_200_list

Step 4: Compute Error

For each simulated reliability:

Error=|Rsim-Rtrue|

Store in error_list

Step 5: Plot Results in a graphical format

Step 6: Stop

12.3. Results and outcomes:

Analytical Reliability: 0.4686620691429672

Fig 12.3.1: Analytical value of reliability of a 2 out of 4 system

TRIALS(N)	Experimental Reliability
100	0.440000
1000	0.454000
10000	0.466800
100000	0.466150
1000000	0.469267

Fig 12.3.2: Experimental value of reliability of a 2 out of 4 system

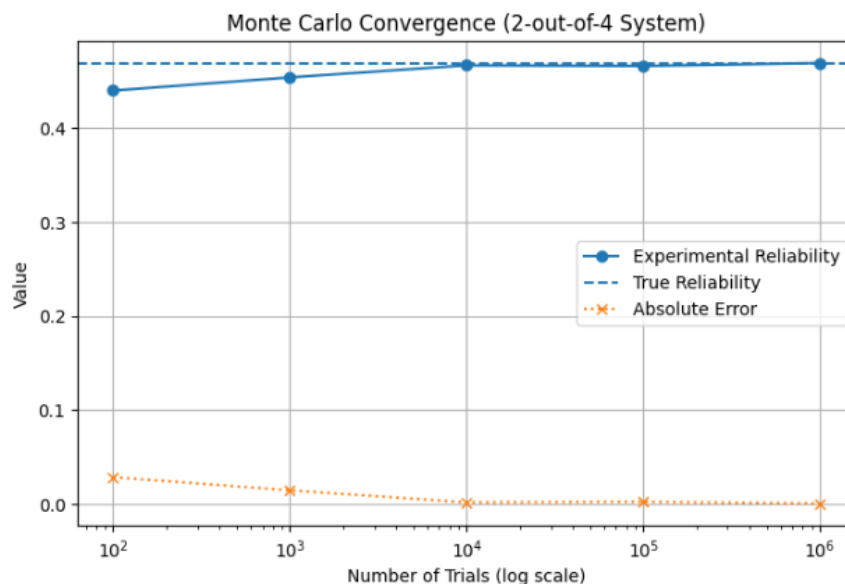


Fig 12.3.3: 2 out of 4 system graph plot

12.4. Observations:

The Monte Carlo simulation results converge to the analytical reliability as the number of trials increases. The error decreases significantly with increasing sample size, confirming the validity of the simulation approach and demonstrating the effect of statistical convergence governed by the Law of Large Numbers.

12.5 Methodology (b):

In a **heterogeneous system**:

- Components are **independent**
- But **NOT identical**
- Each component has a **different failure rate**

Example :

- $\lambda_1=0.002$
- $\lambda_2=0.004$
- $\lambda_3=0.006$
- $\lambda_4=0.008$

System Working Condition

For a **2-out-of-4 system**:

- The system works if **at least 2 components survive**
- It fails if **3 or more components fail**

We consider **all possible combinations** of working/failure states.

Each component can be:

- Working (1)
- Failed (0)

So total combinations: $2^n=16$

Example Combinations (2-out-of-4)

Valid system-working cases:

- 1111 (all working)
- 1110, 1101, 1011, 0111 (3 working)
- 1100, 1010, 1001, 0110, 0101, 0011 (2 working)

Probability of Each State

For any combination:

- Multiply reliabilities of working components
- Multiply failure probabilities of failed components

Example:

For state **1100**: $P = p_1 \cdot p_2 \cdot (1 - p_3) \cdot (1 - p_4)$, here $p = e^{-\lambda t}$

Total System Reliability

Add probabilities of all valid states:

$$R_{\text{system}} = \sum \prod p_i \prod (1 - p_i)$$

Practical Examples:

A heterogeneous system models real-world scenarios like:

- Machines with different qualities
- Components from different manufacturers
- Aging or degraded parts

12.6 Algorithm:

Step 1: Initialize Variables

1. Create empty lists:
 - N_list → to store trial sizes
 - Rel_list → to store simulated reliability
2. Input
 - Failure rates: $\lambda_1, \lambda_2, \lambda_3, \lambda_4$ Mission time: $t = 200$
 - Number of components: $n = 4$
 - System requirement: $k = 2$
 - Trial sizes: $N = [100, 1000, 10000, 100000, 1000000]$

Step 2: Loop Over Trial Sizes

For each N in trial_list:

Step 2.1: Initialize Counter

- Set: system_success = 0

Step 2.2: Perform Monte Carlo Trials

Repeat the following for $i = 1$ to N

Step 2.2.1: Generate Component Lifetimes

For each component $j = 1$ to n :

1. Generate a random number:

$$U_j \sim \text{Uniform}(0, 1)$$

2. Convert to lifetime using exponential distribution: $T_j = -\frac{\ln(1-U_j)}{\lambda_j}$
3. Store T_j in list T

Step 2.2.2: Determine Working Components

- Count number of components satisfying: $T_j \geq t$
- working = $\sum(T_i \geq t)$

Step 2.2.3: Apply k-out-of-n Condition

- If: Working $\geq k$
 → System is **successful**
- Increment: system_success=system_success+1

Step 3. Reliability Estimation

After N trials: Rsim = (Number of successful trials) / N

Step 4 : Compute error: Error=|Rsim-Rtrue|

Step 5 : Plot the graph

Step 6: Stop

12.7. Result and outcomes:

True (Analytical) Reliability: 0.541008

Fig 12.7.1: Analytical value of reliability of a heterogeneous 2 out of 4 system

TRIALS(N)	Experimental Reliability
100	0.500000
1000	0.543000
10000	0.547300
100000	0.538390
1000000	0.541786

Fig 12.7.2: Simulation values of reliability of a heterogeneous 2 out of 4 system

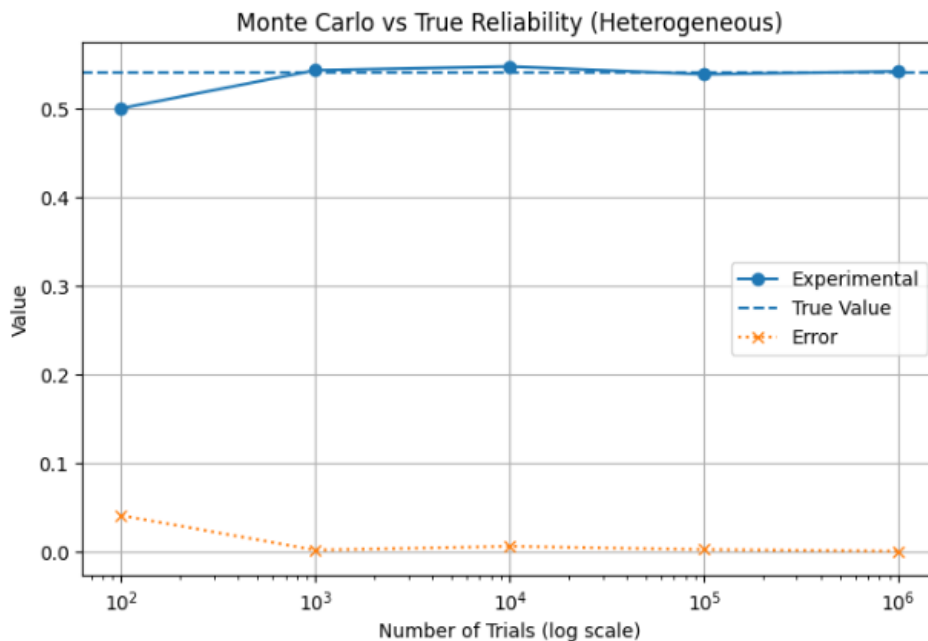


Fig 12.7.3: heterogeneous 2 out of 4 system's graph plot in various number of trials

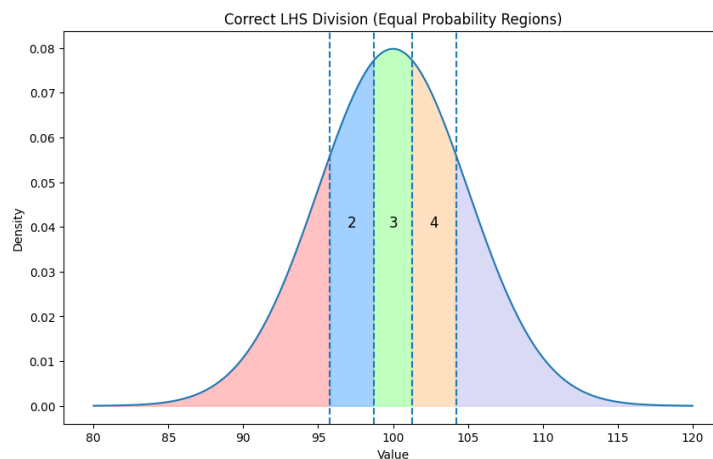
12.8 Observations:

- ☐ Simulated reliability **converges to analytical value** as number of trials increases.
- ☐ **Error decreases** with increase in number of trials.
- ☐ Results are **unstable for small N** but become **stable for large N**.
- ☐ Components with **lower failure rates contribute more** to system reliability.
- ☐ Components with **higher failure rates reduce overall reliability**.
- ☐ System reliability depends on **unequal contribution of components**.
- ☐ Monte Carlo simulation effectively estimates reliability for **non-identical systems**.

12.9 Discussion:

The reliability of a k-out-of-n system was analyzed using analytical and Monte Carlo simulation methods for both homogeneous and heterogeneous cases. In the homogeneous system, analytical results using the binomial model closely matched the simulation results, confirming the validity of the approach. In the heterogeneous system, differing failure rates made analytical evaluation more complex, and reliability was obtained using state enumeration and simulation. The results showed that components contribute unequally to system performance. In both cases, simulation results converged to the true reliability as the number of trials increased, demonstrating the effectiveness of Monte Carlo methods for reliability analysis.

Problem_13: To estimate the reliability of a two-component series system with mixed failure distributions (Weibull and Normal) using Latin Hypercube Sampling and Bernard ranking, and to compute reliability, failure probability, and hazard rate from simulated data.



13.1. Latin Hypercube Sampling:

In real engineering systems:

- Components **do not fail at fixed times**
- Their lifetimes are **uncertain (random)**

So we model them using probability distributions:

- Weibull distribution → flexible failure modeling
- Normal distribution → natural variability

Goal of the analysis

We want to answer: *“How reliable is the system over time?”*

This means finding:

- Probability system survives → $R(t)$
- Probability system fails → $F(t)$
- Failure rate → $h(t)$

Normally, we could use random sampling, but it has problems: **Clustering, Poor coverage**

So we use: Latin Hypercube Sampling

Theory behind LHS

- Divide probability space into equal parts
- Sample once from each part

Ensures: Uniform coverage, Better accuracy ,Faster convergence

Instead of randomly guessing, we do **structured sampling across entire uncertainty range**.

13.2 Algorithm :

Step 1: Initialize

1. Set $N=D \times n$
2. Here we took $N=10000$

Step 2: Generate LHS samples

For each i : $U_i = \frac{(i-1+rand(0,1))}{N}$

Repeat for:

- $U1 \rightarrow$ Component 1
- $U2 \rightarrow$ Component 2

Step 3: Shuffle samples

Randomly shuffle $U1$ and $U2$ ensures independence between components

Step 4: Generate Time-to-Failure

Component 1 (Weibull) : $TTF_1 = \theta \times (-\ln(1 - U1))^{\frac{1}{\beta}}$

Component 2 (Normal) : $TTF_2 = Norm.inverse(U2, mean, std, true)$

Step 5: Compute system failure time

For each sample: $T_{system} = \min(T1, T2)$

Step 6: Compute CDF, Reliability, PDF, hazard rate

Step 7: End

13.3 Results and outcomes:

No of division	5
No of sample in div	2000
beta	1.5
eta	6000
mean	100
st d	5
N	10000

Fig 13.3.1: Assumption of parameters

RAND	TTF1	TTF2	TTF SYSTEM
0.0948	2.68	80.62	2.68
0.3608	15.87	82.21	15.87
...	...	...	...

Fig 13.3.2 LHS component's TTF and system's TTF

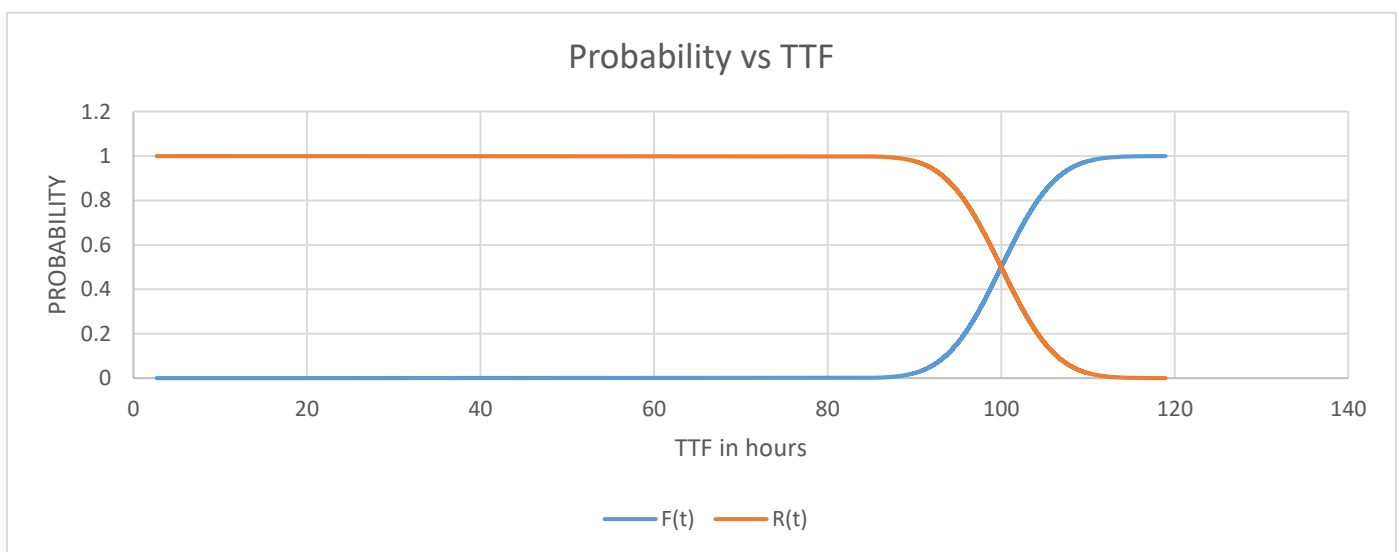


Fig 13.3.3: $F(t)$, $R(t)$ plot vs TTF

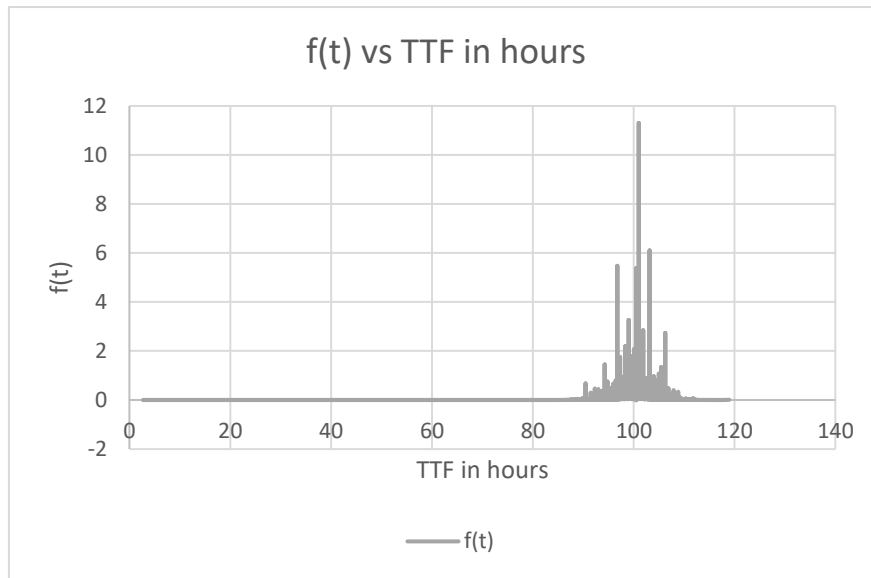


Fig 13.3.4: $f(t)$ plot vs TTF

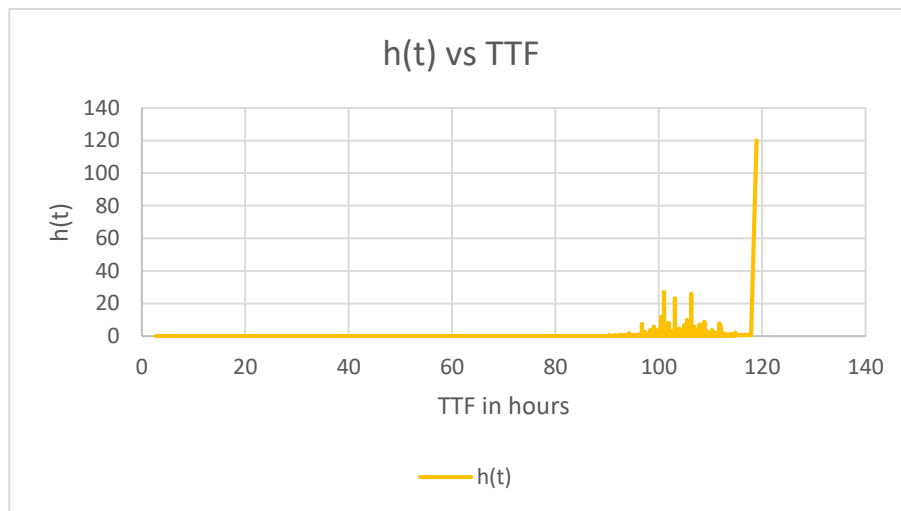


Fig 13.3.5: $h(t)$ plot vs TTF

13.4 Observations:

- The system is a **series system**, so failure is governed by: $T_{\text{system}} = \min(T_1, T_2)$
- Reliability $R(t)$ decreases with time, showing higher failures in early stages.
- The Latin Hypercube Sampling method ensures uniform sampling and improves accuracy.
- The hazard rate indicates **increasing failure tendency over time** (aging effect)

13.5 Discussion:

The reliability of the two-component series system was analyzed using Latin Hypercube Sampling and simulation techniques. The results show that the system failure is primarily governed by the Weibull distribution component, as it fails earlier than the Normal distribution component.

Problem_14: Q1: Stochastic degradation simulation A critical component in an industrial system undergoes degradation over time, modeled as a Gamma process with the following degradation characteristics: The degradation increment $\Delta D(t)$ over a small time interval Δt follows a Gamma distribution with shape parameter $\alpha \Delta t$ and scale parameter β . The degradation at time t is given by $D(t) \sim \text{Gamma}(\alpha t, \beta)$. The component is considered failed when the degradation level reaches a critical threshold D_c . Assume D_c , α , and β .

(a) Simulate 100 degradation paths using the Gamma process with a time step $\Delta t = 1$ and a total observation time of 50 cycles.

(b) Estimate the distribution of the end-of-life (EOL) (i.e., the time at which the component reaches D_c).

(c) Compute the probability that the component survives beyond certain time units/cycles, i.e., compute the reliability function for the system.

14.1. Methodology:

We model degradation using a Gamma stochastic process and use Monte Carlo simulation to estimate system reliability characteristics, including degradation paths, end-of-life distribution, and reliability function.

Model Assumption

$\Delta D(t) \sim \text{Gamma}(\alpha \Delta t, \beta)$

With $\Delta t = 1$, $\Delta D \sim \text{Gamma}(\alpha, \beta)$

Procedure (a) : Simulation of Degradation Paths

Objective: Simulate stochastic degradation behaviour over time.

1. **Initialize parameters:**
 - Shape parameter α
 - Scale parameter β
 - Failure threshold D_c
 - Total time T
 - Number of simulations NNN
2. **Initialize degradation:** $D(0) = 0$
3. **For each simulation ($i = 1$ to N):**
 - For each time step $t = 1$ to T :
 - Generate increment: $\Delta D \sim \text{Gamma}(\alpha, \beta)$
 - Update: $D(t) = D(t-1) + \Delta D$
4. Store degradation paths

Procedure (b) : Estimation of End-of-Life (EOL)

Objective: Determine when the system fails.

Definition

$T_f = \inf \{ t: D(t) \geq D_c \}$

Procedure

1. For each simulated path: Scan degradation over time
2. Identify first time: $D(t) \geq D_c$

Record: $T_f = t \rightarrow$ failure time

3. If threshold not reached within TTT:
 - Mark as **right-censored**
 - Set: $T_f > T$

Procedure (c) : Reliability Function Estimation

Objective: Estimate probability of survival beyond time t

Definition: $R(t) = P(T_f > t)$

Procedure (Simulation-Based)

Case 1: Without censoring

1. For each time t:
 - Count surviving components: $R(t) = (\text{Number of } T_f > t) / N$

Case 2: With censoring

We use: EOL data ,Censoring indicators

1. Apply **Kaplan–Meier estimator**: $R(t) = \prod_{t_i \leq t} (1 - \frac{d_i}{n_i})$

Where:

- d_i = failures at time t_i
- n_i = number at risk

Reliability is estimated as the probability that the system survives beyond time t, using either empirical simulation or Kaplan–Meier estimation

Procedure (d): Maintainability Modelling (MBD-Based)

Objective: To evaluate the probability that a failed system is repaired within a given time.

Maintainability starts **AFTER the curve crosses the threshold**. Once degradation crosses the threshold, the system fails and then repair (maintenance) begins.

Repair Time Model

For each failed component: $Tr,i \sim \text{Exponential}(\lambda)$

System Repair Time : $Tr = \sum_{i=1}^n Tr,i$

Maintainability Function : $M(t)=P(Tr \leq t)$

Procedure:

1. Identify failed systems (from EOL analysis)
2. For each failed system:
 - Assign repair time to each component: $Tr,i \sim \text{Exponential}(\lambda)$
3. Compute total repair time: $Tr = \sum_{i=1}^n Tr,i$
4. Repeat for all simulations
5. Estimate maintainability: $M(t)=(\text{Number of systems repaired by time } t)/N$

Performance Metrics

- **Mean Time To Repair (MTTR):** $MTTR = \frac{1}{N} \sum Tr$

14.2 Algorithm (a): Simulation of Gamma Degradation Paths

Input:

- Shape parameter α
- Scale parameter β
- Failure threshold D_c
- Total time T
- Number of simulations N

Steps:

1. Initialize matrix: $paths[N \times T]=0$
2. For each simulation $i=1$ to N : Set $D=0$
3. For each time step $t=1$ to T :
 - Generate degradation increment: $\Delta D \sim \text{Gamma}(\alpha, \beta)$
 - Update: $D=D+\Delta D$
 - Store: $paths[i,t]=D$
4. Stop

Output: Degradation paths

Algorithm (b): End-of-Life (EOL) Estimation with Censoring

Steps:

1. For each path iii :
 - a. Set failure flag = False
 - b. For each time $t=1$ to T :
 2. If: $D(t) \geq D_c$
→ Store the value : $T_f=t$

- Set failure flag = True
- Break loop

- If failure not observed:
 1. Set:

$T_f = T$

2. Marked as **censored**

Output: EOL values, Censoring indicators

Algorithm c: Reliability Function Estimation

Method 1: Empirical Reliability

1. For each time $t=1$ to T :
 - Count survivors:

$$R(t) = (T_f > t) / N$$

Method 2: Kaplan–Meier (with censoring)

1. Input:
 - EOL
 - Censoring indicator
2. Apply: $R(t) = \prod_{t_i \leq t} (1 - \frac{d_i}{n_i})$

Output: Reliability curve $R(t)$

Algorithm 4: Maintainability Modelling (MBD-Based)

Input:

- Repair rate λ
- Failed systems (from EOL)

Steps:

1. Initialize: Repair time list
2. For each simulation i :
 - a. If system failed: (censored[i] = 0)
 - b. Generate repair time: $T_r \sim \text{Exponential}(\lambda)$
 - c. Store T_r

Algorithm 5: Performance Metrics

Compute:

MTTF=mean (EOL), MTTR=mean(Tr)

$$\text{Availability} = \frac{MTTF}{MTTF + MTTR}$$

14.3 Results and outcomes:

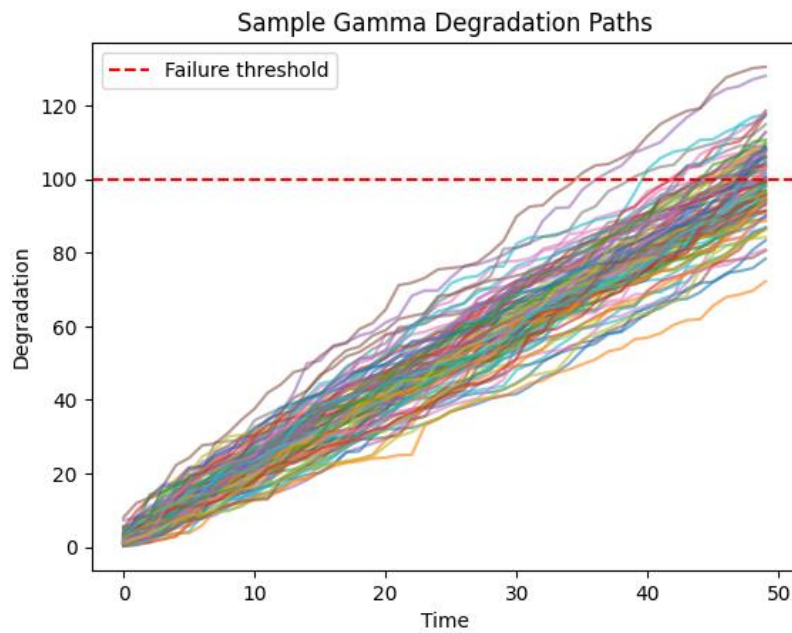


Fig 14.3.1: 100 Gamma degradation paths when observation time 50 cycles

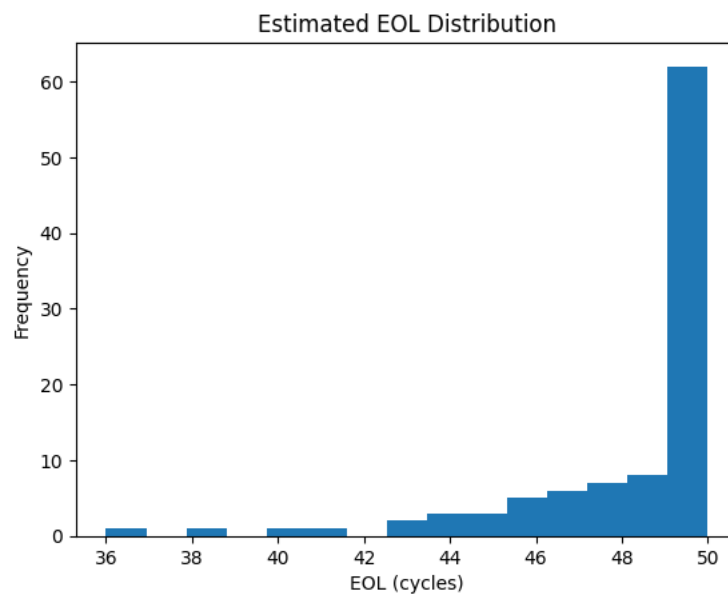


Fig 14.3.2: Histogram of the end-of-life (EOL)

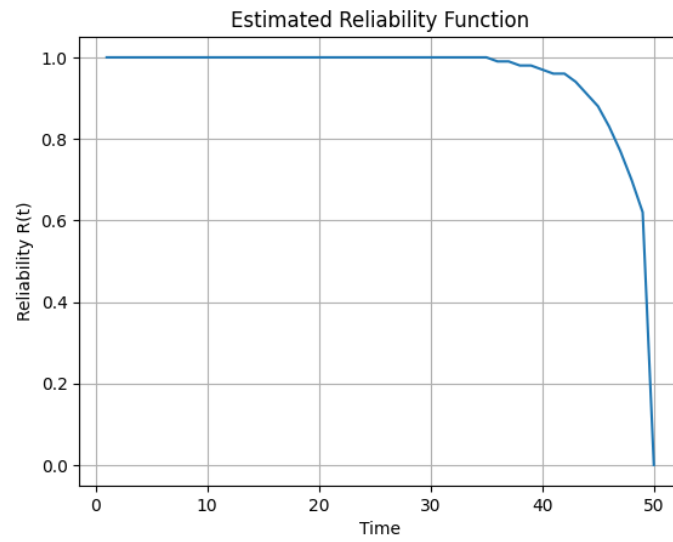


Fig 14.3.2: $R(t)$ vs Time graph

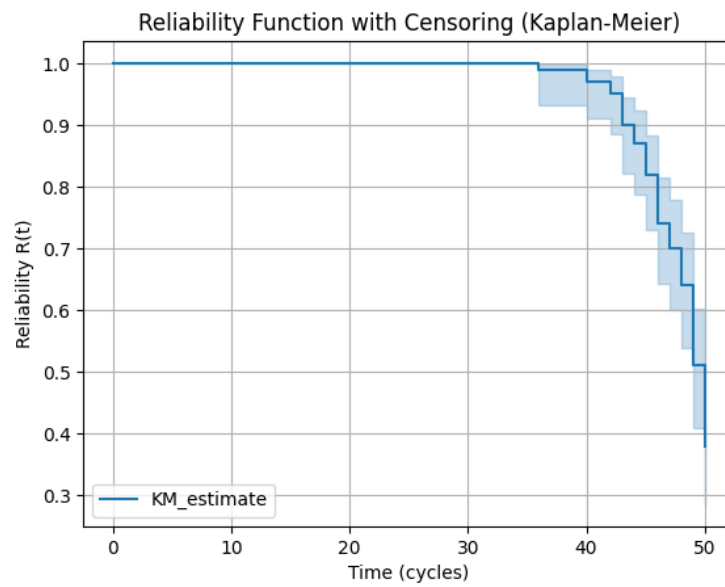


Fig 14.3.3: Reliability function with censoring using KM method

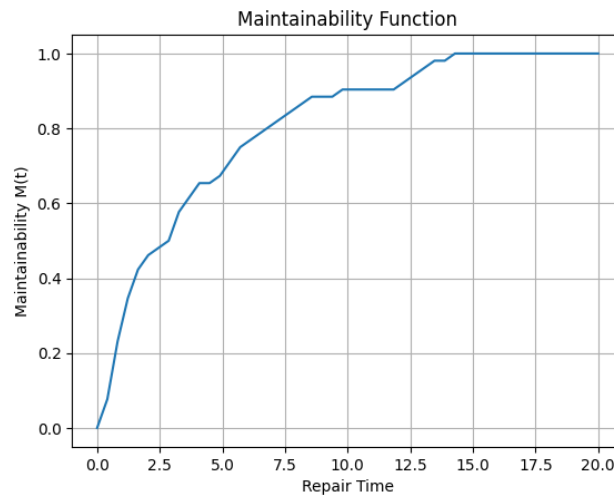


Fig 14.3.4: Maintainability Function of the System

14.4. Observations:

- 1. Degradation Paths:** Degradation increases monotonically over time with variability across different simulation paths. Failure occurs when the degradation crosses the threshold D_c .
- 2. EOL Distribution:** The end-of-life (EOL) shows variability, indicating that failure time is random due to stochastic degradation.
- 3. Empirical Reliability:** Reliability decreases with time, eventually approaching zero, since all components are assumed to fail within the observation period.
- 4. Kaplan–Meier Reliability (with censoring):** Reliability decreases in a stepwise manner and remains above zero, indicating that some components survive beyond the observation time.
- 5. Maintainability Function:** Maintainability increases with repair time and approaches one, showing that most systems can be repaired within a finite time.

14.5. Discussion:

In this study, the degradation behavior of a component was modeled using a Gamma stochastic process, capturing the randomness and cumulative nature of damage over time. The end-of-life (EOL) was determined as the first time the degradation exceeded a critical threshold, demonstrating variability in failure times across simulations. The reliability function showed a decreasing trend over time, reflecting the increasing probability of failure. Incorporating censoring through the Kaplan–Meier method provided a more accurate estimate of reliability by accounting for components that did not fail within the observation period.

Furthermore, maintainability was modeled using an exponential repair time distribution, and the maintainability function indicated that the system can be restored efficiently within a finite time. The combined analysis of reliability and maintainability provides a comprehensive understanding of system performance, highlighting both failure behavior and recovery capability. Overall, the approach demonstrates the effectiveness of stochastic modeling and simulation in reliability and maintenance analysis

Problem_15: Simulation-based estimation of total test-and-repair time in a multi-station facility. A test and repair facility consists of six stations, through which each system must pass sequentially. The time spent at each station is a random variable, and its probability density function (PDF) has been estimated from available data, as shown in the table below. The processing times at different stations are assumed to be statistically independent.

(a) Using Latin Hypercube Sampling (LHS), estimate the total time required to test and repair a randomly chosen system. Compare the results with those obtained using brute-force Monte Carlo simulation (MCS), and demonstrate that the number of samples required for convergence using LHS is significantly lower than that required for brute-force MCS.

(b) Estimate the mean time to repair (MTTR) and the standard deviation of the total test-and-repair time. In addition, determine the times by which 50% and 95% of the systems will have completed the entire test and repair process.

Station number	Probability density function	Parameters
1	$f_1(t_1) = \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}(t_1-10)^2}$	$N(\mu = 10; \sigma = 1)$
2	$f_2(t_2) = \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}(\frac{t_2-20}{\sqrt{2}})^2}$	$N(\mu = 20; \sigma = \sqrt{2})$
3	$f_3(t_3) = \frac{(6)^9}{\Gamma(6)} (t_3)^8 e^{-6 t_3}$	$G(\beta = 9; \eta = 6)$
4	$f_4(t_4) = \frac{1}{\Gamma(10)} (t_4)^9 e^{-t_4}$	$G(\beta = 10; \eta = 1)$
5	$f_5(t_5) = 5 e^{-5 t_5}$	$E(\lambda = 5)$
6	$f_6(t) = \frac{1}{2^5 \Gamma(6)} (t_6)^4 e^{-\frac{1}{2} t_6}$	$\chi^2(\nu = 10)$

N = normal pdf,
 G = gamma pdf,
 E = exponential pdf,
 χ^2 = chi-square pdf,
 μ = mean,
 σ = standard deviation,
 β = shape parameter,
 η = scale parameter,
 ν = degrees of freedom.

15.1 Methodology:

The total test-and-repair time is modeled as the sum of processing times across six independent stations. Random samples are generated using **Monte Carlo Simulation (MCS)** and **Latin Hypercube Sampling (LHS)**.

In MCS, samples are drawn directly from each distribution, while in LHS, the input space is stratified to ensure better coverage and faster convergence.

For each run, total time is computed, and performance measures such as **MTTR (mean)**, **standard deviation**, and **50% and 95% completion times** are estimated. LHS provides more efficient and accurate results compared to MCS.

15.2 Algorithm:

Step 1: Start

Step 2: **Input Parameters** - Define number of samples NNN and specify probability distributions for all six stations.

Step 3: **Generate LHS Samples**

- Divide the interval [0,1] into N equal strata.

- For each station, randomly select one value from each stratum.
- Shuffle the samples to maintain independence between variables.
- Convert uniform samples into actual processing times using inverse CDF of each distribution.

Step 4: **Compute Total Time (LHS)**

- For each sample, sum all station times:

$$T(\text{LHS})=t_1+t_2+t_3+t_4+t_5+t_6$$

Step 5: **Monte Carlo Simulation (MCS)**

- Generate random samples directly from each station's distribution.
- Compute total time for each run:

$$T(\text{MCS})=t_1+t_2+t_3+t_4+t_5+t_6$$

Step 6: **Statistical Analysis**

- Estimate mean (MTTR), standard deviation, and percentiles.
- Plot histograms of $T(\text{LHS})$ and $T(\text{MCS})$

Step 7: **Comparison**

- Compare both methods in terms of spread and smoothness.
- Observe that LHS provides better coverage and faster convergence than MCS.

Step 8: End

15.4 **Result and outcome:**

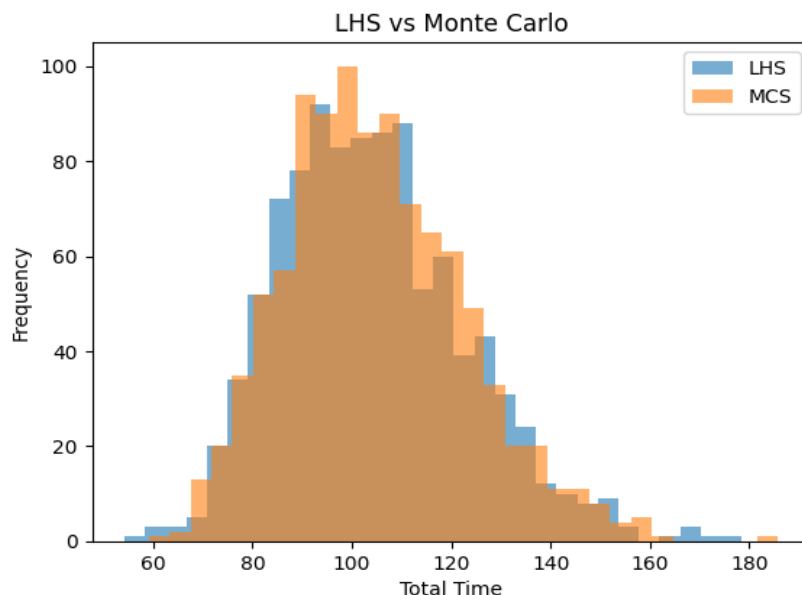


Fig 15.4.1: An overlaid histogram used to compare the frequency distributions of total time obtained from LHS and Monte Carlo simulations.

Mean (MTTR): 104.19013969532679
Standard Deviation: 18.607917906964044
50% completion time: 102.70566524706771
95% completion time: 136.36241396570801

Fig 15.4.2: Statistical Summary of Total Test-and-Repair Time

15.5 Observation:

The mean total test-and-repair time (MTTR) is approximately **104 units**, indicating the average time required for system completion. The standard deviation (~18.6) shows moderate variability in processing time across different simulations. The 50% completion time (~102.7) is close to the mean, suggesting a fairly symmetric distribution. The 95% completion time (~136.4) indicates that most systems are completed within this time, reflecting the upper bound of expected delays

CONCLUSION

Monte Carlo simulation is a powerful tool for analysing probabilistic systems such as coin tosses, dice rolls, and reliability models. In addition to estimating event probabilities, it is widely used to study series and parallel system reliability, capacity versus demand behaviour, system integration, Stochastic degradation and performance measures such as Mean Time to Failure (MTTF). This report demonstrates simulation-based probability estimation, convergence analysis, and error evaluation to validate theoretical concepts.

Reference: 1) Reliability Engineering Author: Elsayed A. — Mohamed A. Elsayed

2) Reliability and Life Testing Handbook Authors: *Dimitri Kececioglu*

4) Applied Reliability Authors: *John D. Andrews & Walter R. Moss*

5) Simulation Modeling and Analysis Authors: *A. M. Law & W. D. Kelton*

7) An Introduction to Reliability and Maintainability Engineering Author: *Charles E. Ebeling*

8) NIST/SEMATECH e-Handbook of Statistical Methods• Covers probability distributions, reliability concepts <https://www.itl.nist.gov/div898/handbook/>

9) MathWorks Reliability & Monte Carlo Tutorials Applied simulation examples <https://www.mathworks.com>